

```

1 struct sock {
2     struct sock_common    __sk_common; /* early demux fields */
3     struct dst_entry __rcu *sk_rx_dst;
4     int                   sk_rx_dst_ifindex;
5     u32                    sk_rx_dst_cookie;
6     socket_lock_t         sk_lock;
7     atomic_t              sk_drops;
8     int                    sk_rcvlowat;
9     struct sk_buff_head    sk_error_queue;
10    struct sk_buff_head    sk_receive_queue;
11    struct {
12        atomic_t           rmem_alloc;
13        int                len;
14        struct sk_buff     *head;
15        struct sk_buff     *tail;
16    } sk_backlog;
17    int                     sk_forward_alloc;
18    u32                     sk_reserved_mem;
19    #ifdef CONFIG_NET_RX_BUSY_POLL
20    unsigned int            sk_ll_usec;
21    /* ===== mostly read cache line ===== */
22    unsigned int            sk_napi_id;
23    #endif
24    int                     sk_rcvbuf;
25    int                     sk_disconnects;
26    struct sk_filter __rcu *sk_filter;
27    union {
28        struct socket_wq __rcu *sk_wq;
29        /* private: */
30        struct socket_wq    *sk_wq_raw;
31        /* public: */
32    };
33    #ifdef CONFIG_XFRM
34    struct xfrm_policy __rcu *sk_policy[2];
35    #endif
36    struct dst_entry __rcu *sk_dst_cache;
37    atomic_t           sk_omem_alloc;
38    int                 sk_sndbuf;
39    /* ===== cache line for TX ===== */
40    int                 sk_wmem_queued;
41    refcount_t          sk_wmem_alloc;
42    unsigned long       sk_tsq_flags;
43    union {
44        struct sk_buff *sk_send_head;
45        struct rb_root tcp_rtx_queue;
46    };
47    struct sk_buff_head sk_write_queue;
48    __s32               sk_peek_off;
49    int                 sk_write_pending;
50    __u32               sk_dst_pending_confirm;
51    u32                 sk_pacing_status; /* see enum sk_pacing */
52    long                sk_sndtimeo;
53    struct timer_list   sk_timer;
54    __u32               sk_priority;
55    __u32               sk_mark;
56    unsigned long       sk_pacing_rate; /* bytes per second */
57    unsigned long       sk_max_pacing_rate;
58    struct page_frag    sk_frag;
59    netdev_features_t   sk_route_caps;
60    int                 sk_gso_type;
61    unsigned int        sk_gso_max_size;
62    gfp_t               sk_allocation;
63    __u32               sk_txhash;
64    /*
65     * Because of non atomicity rules, all
66     * changes are protected by socket lock.
67     */
68    u8                  sk_gso_disabled : 1,
69    sk_kern_sock : 1,
70    sk_no_check_tx : 1,

```

Ответы на билеты (односторонняя печать)

Эта версия нужна для файла Печать. tур Всего страниц: 287 Билеты + вопросы в конце Экспортируйте, выключив Tagged PDF, чтобы уменьшить размер файла. Нажмите VerFlase на старой версии PDF, чтобы обновить её новой	Содержание 1 Управление типами устройств, специализированные файлы устройств, адресные типичные устройства и их спецификации в системе (тип dev. 1, dev. 2, dev. 3, dev. 4, dev. 5, dev. 6, dev. 7, dev. 8, dev. 9, dev. 10, dev. 11, dev. 12, dev. 13, dev. 14, dev. 15, dev. 16, dev. 17, dev. 18, dev. 19, dev. 20, dev. 21, dev. 22, dev. 23, dev. 24, dev. 25, dev. 26, dev. 27, dev. 28, dev. 29, dev. 30, dev. 31, dev. 32, dev. 33, dev. 34, dev. 35, dev. 36, dev. 37, dev. 38, dev. 39, dev. 40, dev. 41, dev. 42, dev. 43, dev. 44, dev. 45, dev. 46, dev. 47, dev. 48, dev. 49, dev. 50, dev. 51, dev. 52, dev. 53, dev. 54, dev. 55, dev. 56, dev. 57, dev. 58, dev. 59, dev. 60, dev. 61, dev. 62, dev. 63, dev. 64, dev. 65, dev. 66, dev. 67, dev. 68, dev. 69, dev. 70, dev. 71, dev. 72, dev. 73, dev. 74, dev. 75, dev. 76, dev. 77, dev. 78, dev. 79, dev. 80, dev. 81, dev. 82, dev. 83, dev. 84, dev. 85, dev. 86, dev. 87, dev. 88, dev. 89, dev. 90, dev. 91, dev. 92, dev. 93, dev. 94, dev. 95, dev. 96, dev. 97, dev. 98, dev. 99, dev. 100, dev. 101, dev. 102, dev. 103, dev. 104, dev. 105, dev. 106, dev. 107, dev. 108, dev. 109, dev. 110, dev. 111, dev. 112, dev. 113, dev. 114, dev. 115, dev. 116, dev. 117, dev. 118, dev. 119, dev. 120, dev. 121, dev. 122, dev. 123, dev. 124, dev. 125, dev. 126, dev. 127, dev. 128, dev. 129, dev. 130, dev. 131, dev. 132, dev. 133, dev. 134, dev. 135, dev. 136, dev. 137, dev. 138, dev. 139, dev. 140, dev. 141, dev. 142, dev. 143, dev. 144, dev. 145, dev. 146, dev. 147, dev. 148, dev. 149, dev. 150, dev. 151, dev. 152, dev. 153, dev. 154, dev. 155, dev. 156, dev. 157, dev. 158, dev. 159, dev. 160, dev. 161, dev. 162, dev. 163, dev. 164, dev. 165, dev. 166, dev. 167, dev. 168, dev. 169, dev. 170, dev. 171, dev. 172, dev. 173, dev. 174, dev. 175, dev. 176, dev. 177, dev. 178, dev. 179, dev. 180, dev. 181, dev. 182, dev. 183, dev. 184, dev. 185, dev. 186, dev. 187, dev. 188, dev. 189, dev. 190, dev. 191, dev. 192, dev. 193, dev. 194, dev. 195, dev. 196, dev. 197, dev. 198, dev. 199, dev. 200, dev. 201, dev. 202, dev. 203, dev. 204, dev. 205, dev. 206, dev. 207, dev. 208, dev. 209, dev. 210, dev. 211, dev. 212, dev. 213, dev. 214, dev. 215, dev. 216, dev. 217, dev. 218, dev. 219, dev. 220, dev. 221, dev. 222, dev. 223, dev. 224, dev. 225, dev. 226, dev. 227, dev. 228, dev. 229, dev. 230, dev. 231, dev. 232, dev. 233, dev. 234, dev. 235, dev. 236, dev. 237, dev. 238, dev. 239, dev. 240, dev. 241, dev. 242, dev. 243, dev. 244, dev. 245, dev. 246, dev. 247, dev. 248, dev. 249, dev. 250, dev. 251, dev. 252, dev. 253, dev. 254, dev. 255, dev. 256, dev. 257, dev. 258, dev. 259, dev. 260, dev. 261, dev. 262, dev. 263, dev. 264, dev. 265, dev. 266, dev. 267, dev. 268, dev. 269, dev. 270, dev. 271, dev. 272, dev. 273, dev. 274, dev. 275, dev. 276, dev. 277, dev. 278, dev. 279, dev. 280, dev. 281, dev. 282, dev. 283, dev. 284, dev. 285, dev. 286, dev. 287, dev. 288, dev. 289, dev. 290, dev. 291, dev. 292, dev. 293, dev. 294, dev. 295, dev. 296, dev. 297, dev. 298, dev. 299, dev. 300, dev. 301, dev. 302, dev. 303, dev. 304, dev. 305, dev. 306, dev. 307, dev. 308, dev. 309, dev. 310, dev. 311, dev. 312, dev. 313, dev. 314, dev. 315, dev. 316, dev. 317, dev. 318, dev. 319, dev. 320, dev. 321, dev. 322, dev. 323, dev. 324, dev. 325, dev. 326, dev. 327, dev. 328, dev. 329, dev. 330, dev. 331, dev. 332, dev. 333, dev. 334, dev. 335, dev. 336, dev. 337, dev. 338, dev. 339, dev. 340, dev. 341, dev. 342, dev. 343, dev. 344, dev. 345, dev. 346, dev. 347, dev. 348, dev. 349, dev. 350, dev. 351, dev. 352, dev. 353, dev. 354, dev. 355, dev. 356, dev. 357, dev. 358, dev. 359, dev. 360, dev. 361, dev. 362, dev. 363, dev. 364, dev. 365, dev. 366, dev. 367, dev. 368, dev. 369, dev. 370, dev. 371, dev. 372, dev. 373, dev. 374, dev. 375, dev. 376, dev. 377, dev. 378, dev. 379, dev. 380, dev. 381, dev. 382, dev. 383, dev. 384, dev. 385, dev. 386, dev. 387, dev. 388, dev. 389, dev. 390, dev. 391, dev. 392, dev. 393, dev. 394, dev. 395, dev. 396, dev. 397, dev. 398, dev. 399, dev. 400, dev. 401, dev. 402, dev. 403, dev. 404, dev. 405, dev. 406, dev. 407, dev. 408, dev. 409, dev. 410, dev. 411, dev. 412, dev. 413, dev. 414, dev. 415, dev. 416, dev. 417, dev. 418, dev. 419, dev. 420, dev. 421, dev. 422, dev. 423, dev. 424, dev. 425, dev. 426, dev. 427, dev. 428, dev. 429, dev. 430, dev. 431, dev. 432, dev. 433, dev. 434, dev. 435, dev. 436, dev. 437, dev. 438, dev. 439, dev. 440, dev. 441, dev. 442, dev. 443, dev. 444, dev. 445, dev. 446, dev. 447, dev. 448, dev. 449, dev. 450, dev. 451, dev. 452, dev. 453, dev. 454, dev. 455, dev. 456, dev. 457, dev. 458, dev. 459, dev. 460, dev. 461, dev. 462, dev. 463, dev. 464, dev. 465, dev. 466, dev. 467, dev. 468, dev. 469, dev. 470, dev. 471, dev. 472, dev. 473, dev. 474, dev. 475, dev. 476, dev. 477, dev. 478, dev. 479, dev. 480, dev. 481, dev. 482, dev. 483, dev. 484, dev. 485, dev. 486, dev. 487, dev. 488, dev. 489, dev. 490, dev. 491, dev. 492, dev. 493, dev. 494, dev. 495, dev. 496, dev. 497, dev. 498, dev. 499, dev. 500, dev. 501, dev. 502, dev. 503, dev. 504, dev. 505, dev. 506, dev. 507, dev. 508, dev. 509, dev. 510, dev. 511, dev. 512, dev. 513, dev. 514, dev. 515, dev. 516, dev. 517, dev. 518, dev. 519, dev. 520, dev. 521, dev. 522, dev. 523, dev. 524, dev. 525, dev. 526, dev. 527, dev. 528, dev. 529, dev. 530, dev. 531, dev. 532, dev. 533, dev. 534, dev. 535, dev. 536, dev. 537, dev. 538, dev. 539, dev. 540, dev. 541, dev. 542, dev. 543, dev. 544, dev. 545, dev. 546, dev. 547, dev. 548, dev. 549, dev. 550, dev. 551, dev. 552, dev. 553, dev. 554, dev. 555, dev. 556, dev. 557, dev. 558, dev. 559, dev. 560, dev. 561, dev. 562, dev. 563, dev. 564, dev. 565, dev. 566, dev. 567, dev. 568, dev. 569, dev. 57
--	---

```

71     sk_no_check_rx : 1,
72     sk_userlocks : 4;
73     u8                sk_pacing_shift;
74     u16               sk_type;
75     u16               sk_protocol;
76     u16               sk_gso_max_segs;
77     unsigned long     sk_lingertime;
78     struct proto      *sk_prot_creator;
79     rwlock_t          sk_callback_lock;
80     int               sk_err,
81     sk_err_soft;
82     u32               sk_ack_backlog;
83     u32               sk_max_ack_backlog;
84     kuid_t            sk_uid;
85     u8                sk_txrehash;
86     #ifdef CONFIG_NET_RX_BUSY_POLL
87     u8                sk_prefer_busy_poll;
88     u16               sk_busy_poll_budget;
89     #endif
90     spinlock_t         sk_peer_lock;
91     int               sk_bind_phc;
92     struct pid         *sk_peer_pid;
93     const struct cred  *sk_peer_cred;
94     long               sk_rcvtimeo;
95     ktime_t            sk_stamp;
96     #if BITS_PER_LONG==32
97     seqlock_t          sk_stamp_seq;
98     #endif
99     atomic_t           sk_tskey;
100    atomic_t           sk_zckey;
101    u32                sk_tsflags;
102    u8                 sk_shutdown;
103    u8                 sk_clockid;
104    u8                 sk_txtime_deadline_mode : 1,
105    sk_txtime_report_errors : 1,
106    sk_txtime_unused : 6;
107    bool               sk_use_task_frag;
108    struct socket      *sk_socket;
109    void               *sk_user_data;
110    #ifdef CONFIG_SECURITY
111    void               *sk_security;
112    #endif
113    struct sock_cgroup_data sk_cgrp_data;
114    struct mem_cgroup  *sk_memcg;
115    void               (*sk_state_change)(struct sock *sk);
116    void               (*sk_data_ready)(struct sock *sk);
117    void               (*sk_write_space)(struct sock *sk);
118    void               (*sk_error_report)(struct sock *sk);
119    int                (*sk_backlog_rcv)(struct sock *sk,
120    struct sk_buff *skb);
121    #ifdef CONFIG_SOCK_VALIDATE_XMIT
122    struct sk_buff*     (*sk_validate_xmit_skb)(struct sock *sk,
123    struct net_device *dev,
124    struct sk_buff *skb);
125    #endif
126    void               (*sk_destruct)(struct sock *sk);
127    struct sock_reuseport __rcu *sk_reuseport_cb;
128    #ifdef CONFIG_BPF_SYSCALL
129    struct bpf_local_storage __rcu *sk_bpf_storage;
130    #endif
131    struct rcu_head     sk_rcu;
132    netns_tracker       ns_tracker;
133 };

```



```

1 struct socket {
2     socket_state      state;
3     short             type;
4     unsigned long     flags;
5     struct file       *file;
6     struct sock       *sk;
7     const struct proto_ops *ops; /* Might change with IPV6_ADDRFORM or MPTCP. */
8     struct socket_wq  wq;
9 };

```

```

1 struct sockaddr {
2     sa_family_t      sa_family; /* address family, AF_xxx */
3     char sa_data_min[14];
4 };
5
6 struct in_addr {
7     u32 s_addr;
8 };
9
10 struct sockaddr_in {
11     __kernel_sa_family_t sin_family; /* Address family */
12     __be16 sin_port; /* Port number */
13     struct in_addr sin_addr; /* Internet address*/
14
15     /* Pad to size of 'struct sockaddr'. */
16     unsigned char __pad[__SOCK_SIZE__ - sizeof(short int) - sizeof(unsigned short int) -
17     sizeof(struct in_addr)];
17 };

```

```

1 struct epoll_event {
2     __poll_t events;
3     __u64 data;
4 };

```



```

1 struct super_block {
2     struct list_head      s_list;           /* Keep this first */
3     dev_t                 s_dev;           /* search index; _not_ kdev_t */
4     unsigned char         s_blocksize_bits;
5     unsigned long         s_blocksize;
6     loff_t                s_maxbytes;      /* Max file size */
7     struct file_system_type *s_type;
8     const struct super_operations *s_op;
9     const struct dquot_operations *dq_op;
10    const struct quotactl_ops *s_qcop;
11    const struct export_operations *s_export_op;
12    unsigned long          s_flags;
13    unsigned long          s_iflags;        /* internal SB_I_* flags */
14    unsigned long          s_magic;
15    struct dentry           *s_root;
16    struct rw_semaphore     s_umount;
17    int                    s_count;
18    atomic_t               s_active;
19    #ifdef CONFIG_SECURITY
20    void                   *s_security;
21    #endif
22    const struct xattr_handler * const *s_xattr;
23    #ifdef CONFIG_FS_ENCRYPTION
24    const struct fscrypt_operations *s_cop;
25    struct fscrypt_keyring *s_master_keys; /* master crypto keys in use */
26    #endif
27    #ifdef CONFIG_FS_VERITY
28    const struct fsverity_operations *s_vop;
29    #endif
30    #if IS_ENABLED(CONFIG_UNICODE)
31    struct unicode_map *s_encoding;
32    __u16 s_encoding_flags;
33    #endif
34    struct hlist_bl_head    s_roots;         /* alternate root dentries for NFS */
35    struct list_head        s_mounts;        /* list of mounts; _not_ for fs use */
36    struct block_device     *s_bdev;
37    struct bdev_handle      *s_bdev_handle;
38    struct backing_dev_info *s_bdi;
39    struct mtd_info         *s_mtd;
40    struct hlist_node       s_instances;
41    unsigned int            s_quota_types;    /* Bitmask of supported quota types */
42    struct quota_info        s_dquot;        /* Diskquota specific options */
43
44    struct sb_writers        s_writers;
45
46    /*
47     * Keep s_fs_info, s_time_gran, s_fsnotify_mask, and
48     * s_fsnotify_marks together for cache efficiency. They are frequently
49     * accessed and rarely modified.
50     */
51    void                   *s_fs_info;      /* Filesystem private info */
52
53    /* Granularity of c/m/ctime in ns (cannot be worse than a second) */
54    u32                    s_time_gran;
55    /* Time limits for c/m/ctime in seconds */
56    time64_t               s_time_min;
57    time64_t               s_time_max;
58    #ifdef CONFIG_FSNOTIFY
59    __u32                   s_fsnotify_mask;
60    struct fsnotify_mark_connector __rcu *s_fsnotify_marks;
61    #endif
62
63    char                   s_id[32];        /* Informational name */
64    uuid_t                 s_uuid;         /* UUID */
65
66    unsigned int            s_max_links;
67
68    /*
69     * The next field is for VFS *only*. No filesystems have any business
70     * even looking at it. You had been warned.

```



```

71  */
72  struct mutex s_vfs_rename_mutex;          /* Kludge */
73
74  /*
75  * Filesystem subtype.  If non-empty the filesystem type field
76  * in /proc/mounts will be "type.subtype"
77  */
78  const char *s_subtype;
79
80  const struct dentry_operations *s_d_op; /* default d_op for dentries */
81
82  struct shrinker *s_shrink;                /* per-sb shrinker handle */
83
84  /* Number of inodes with nlink == 0 but still referenced */
85  atomic_long_t s_remove_count;
86
87  /*
88  * Number of inode/mount/sb objects that are being watched, note that
89  * inodes objects are currently double-accounted.
90  */
91  atomic_long_t s_fsnotify_connectors;
92
93  /* Read-only state of the superblock is being changed */
94  int s_readonly_remount;
95
96  /* per-sb errseq_t for reporting writeback errors via syncfs */
97  errseq_t s_wb_err;
98
99  /* AIO completions deferred from interrupt context */
100 struct workqueue_struct *s_dio_done_wq;
101 struct hlist_head s_pins;
102
103 /*
104 * Owning user namespace and default context in which to
105 * interpret filesystem uids, gids, quotas, device nodes,
106 * xattrs and security labels.
107 */
108 struct user_namespace *s_user_ns;
109
110 /*
111 * The list_lru structure is essentially just a pointer to a table
112 * of per-node lru lists, each of which has its own spinlock.
113 * There is no need to put them into separate cachelines.
114 */
115 struct list_lru          s_dentry_lru;
116 struct list_lru          s_inode_lru;
117 struct rcu_head          rcu;
118 struct work_struct       destroy_work;
119
120 struct mutex             s_sync_lock;      /* sync serialisation lock */
121
122 /*
123 * Indicates how deep in a filesystem stack this SB is
124 */
125 int s_stack_depth;
126
127 /* s_inode_list_lock protects s_inodes */
128 spinlock_t              s_inode_list_lock ____cacheline_aligned_in_smp;
129 struct list_head         s_inodes;        /* all inodes */
130
131 spinlock_t              s_inode_wblist_lock;
132 struct list_head         s_inodes_wb;     /* writeback inodes */
133 } __randomize_layout;

```



```

1 struct super_operations {
2     struct inode *(*alloc_inode)(struct super_block *sb);
3     void (*destroy_inode)(struct inode *);
4     void (*free_inode)(struct inode *);
5
6     void (*dirty_inode) (struct inode *, int flags);
7     int (*write_inode) (struct inode *, struct writeback_control *wbc);
8     int (*drop_inode) (struct inode *);
9     void (*evict_inode) (struct inode *);
10    void (*put_super) (struct super_block *);
11    int (*sync_fs)(struct super_block *sb, int wait);
12    int (*freeze_super) (struct super_block *, enum freeze_holder who);
13    int (*freeze_fs) (struct super_block *);
14    int (*thaw_super) (struct super_block *, enum freeze_holder who);
15    int (*unfreeze_fs) (struct super_block *);
16    int (*statfs) (struct dentry *, struct kstatfs *);
17    int (*remount_fs) (struct super_block *, int *, char *);
18    void (*umount_begin) (struct super_block *);
19
20    int (*show_options)(struct seq_file *, struct dentry *);
21    int (*show_devname)(struct seq_file *, struct dentry *);
22    int (*show_path)(struct seq_file *, struct dentry *);
23    int (*show_stats)(struct seq_file *, struct dentry *);
24    #ifdef CONFIG_QUOTA
25    ssize_t (*quota_read)(struct super_block *, int, char *, size_t, loff_t);
26    ssize_t (*quota_write)(struct super_block *, int, const char *, size_t, loff_t);
27    struct dquot __rcu **(*get_dquots)(struct inode *);
28    #endif
29    long (*nr_cached_objects)(struct super_block *,
30    struct shrink_control *);
31    long (*free_cached_objects)(struct super_block *,
32    struct shrink_control *);
33    void (*shutdown)(struct super_block *sb);
34 };

```



```

1 struct dentry {
2     /* RCU lookup touched fields */
3     unsigned int d_flags;           /* protected by d_lock */
4     seqcount_spinlock_t d_seq;     /* per dentry seqlock */
5     struct hlist_bl_node d_hash;   /* lookup hash list */
6     struct dentry *d_parent;       /* parent directory */
7     struct qstr d_name;
8     struct inode *d_inode;         /* Where the name belongs to - NULL is
9     * negative */
10    unsigned char d_iname[DNAME_INLINE_LEN]; /* small names */
11
12    /* Ref lookup also touches following */
13    struct lockref d_lockref;       /* per-dentry lock and refcount */
14    const struct dentry_operations *d_op;
15    struct super_block *d_sb;       /* The root of the dentry tree */
16    unsigned long d_time;           /* used by d_revalidate */
17    void *d_fsdata;                /* fs-specific data */
18
19    union {
20        struct list_head d_lru;     /* LRU list */
21        wait_queue_head_t *d_wait; /* in-lookup ones only */
22    };
23    struct hlist_node d_sib;         /* child of parent list */
24    struct hlist_head d_children;    /* our children */
25    /*
26     * d_alias and d_rcu can share memory
27     */
28    union {
29        struct hlist_node d_alias;  /* inode alias list */
30        struct hlist_bl_node d_in_lookup_hash; /* only for in-lookup ones */
31        struct rcu_head d_rcu;
32    } d_u;
33 };

```

```

1 struct dentry_operations {
2     int (*d_revalidate)(struct dentry *, unsigned int);
3     int (*d_weak_revalidate)(struct dentry *, unsigned int);
4     int (*d_hash)(const struct dentry *, struct qstr *);
5     int (*d_compare)(const struct dentry *,
6     unsigned int, const char *, const struct qstr *);
7     int (*d_delete)(const struct dentry *);
8     int (*d_init)(struct dentry *);
9     void (*d_release)(struct dentry *);
10    void (*d_prune)(struct dentry *);
11    void (*d_iput)(struct dentry *, struct inode *);
12    char *(*d_dname)(struct dentry *, char *, int);
13    struct vfsmount *(*d_automount)(struct path *);
14    int (*d_manage)(const struct path *, bool);
15    struct dentry *(*d_real)(struct dentry *, const struct inode *);
16 } ____cacheline_aligned;

```



```

1 struct inode {
2     umode_t                i_mode;
3     unsigned short         i_opflags;
4     kuid_t                 i_uid;
5     kgid_t                 i_gid;
6     unsigned int           i_flags;
7     #ifdef CONFIG_FS_POSIX_ACL
8     struct posix_acl        *i_acl;
9     struct posix_acl        *i_default_acl;
10    #endif
11    const struct inode_operations *i_op;
12    struct super_block       *i_sb;
13    struct address_space     *i_mapping;
14    #ifdef CONFIG_SECURITY
15    void                     *i_security;
16    #endif
17    /* Stat data, not accessed from path walking */
18    unsigned long            i_ino;
19    /* Filesystems may only read i_nlink directly. */
20    union {
21        const unsigned int i_nlink;
22        unsigned int __i_nlink;
23    };
24    dev_t                    i_rdev;
25    loff_t                    i_size;
26    struct timespec64        __i_atime;
27    struct timespec64        __i_mtime;
28    struct timespec64        __i_ctime; /* use inode*_ctime accessors! */
29    spinlock_t               i_lock; /* i_blocks, i_bytes, maybe i_size */
30    unsigned short           i_bytes;
31    u8                        i_blkbits;
32    u8                        i_write_hint;
33    blkcnt_t                 i_blocks;
34    #ifdef __NEED_I_SIZE_ORDERED
35    seqcount_t                i_size_seqcount;
36    #endif
37    /* Misc */
38    unsigned long             i_state;
39    struct rw_semaphore       i_rwsem;
40    unsigned long             dirtied_when; /* jiffies of first dirtying */
41    unsigned long             dirtied_time_when;
42    struct hlist_node         i_hash;
43    struct list_head          i_io_list; /* backing dev IO list */
44    #ifdef CONFIG_CGROUP_WRITEBACK
45    struct bdi_writeback      *i_wb; /* the associated cgroup wb */
46    /* foreign inode detection, see wbc_detach_inode() */
47    int                       i_wb_frn_winner;
48    u16                       i_wb_frn_avg_time;
49    u16                       i_wb_frn_history;
50    #endif
51    struct list_head          i_lru; /* inode LRU list */
52    struct list_head          i_sb_list;
53    struct list_head          i_wb_list; /* backing dev writeback list */
54    union {
55        struct hlist_head     i_dentry;
56        struct rcu_head        i_rcu;
57    };
58    atomic64_t                i_version;
59    atomic64_t                i_sequence; /* see futex */
60    atomic_t                  i_count;
61    atomic_t                  i_dio_count;
62    atomic_t                  i_writecount;
63    #if defined(CONFIG_IMA) || defined(CONFIG_FILE_LOCKING)
64    atomic_t                  i_readcount; /* struct files open R0 */
65    #endif
66    union {
67        const struct file_operations *i_fop; /* former ->i_op->default_file_ops */
68        void (*free_inode)(struct inode *);
69    };
70    struct file_lock_context  *i_flctx;

```



```

71 struct address_space      i_data;
72 struct list_head          i_devices;
73 union {
74     struct pipe_inode_info *i_pipe;
75     struct cdev            *i_cdev;
76     char                   *i_link;
77     unsigned               i_dir_seq;
78 };
79 __u32                      i_generation;
80 #ifdef CONFIG_FSNOTIFY
81 __u32                      i_fsnotify_mask; /* all events this inode cares about */
82 struct fsnotify_mark_connector __rcu    *i_fsnotify_marks;
83 #endif
84 #ifdef CONFIG_FS_ENCRYPTION
85 struct fscrypt_inode_info    *i_crypt_info;
86 #endif
87 #ifdef CONFIG_FS_VERITY
88 struct fsverity_info         *i_verity_info;
89 #endif
90 void                        *i_private; /* fs or device private pointer */
91 } __randomize_layout;

```

```

1 struct inode_operations {
2     struct dentry * (*lookup) (struct inode *, struct dentry *, unsigned int);
3     const char * (*get_link) (struct dentry *, struct inode *, struct delayed_call *);
4     int (*permission) (struct mnt_idmap *, struct inode *, int);
5     struct posix_acl * (*get_inode_acl) (struct inode *, int, bool);
6
7     int (*readlink) (struct dentry *, char __user *, int);
8
9     int (*create) (struct mnt_idmap *, struct inode *, struct dentry *,
10     umode_t, bool);
11     int (*link) (struct dentry *, struct inode *, struct dentry *);
12     int (*unlink) (struct inode *, struct dentry *);
13     int (*symlink) (struct mnt_idmap *, struct inode *, struct dentry *,
14     const char *);
15     int (*mkdir) (struct mnt_idmap *, struct inode *, struct dentry *,
16     umode_t);
17     int (*rmdir) (struct inode *, struct dentry *);
18     int (*mknod) (struct mnt_idmap *, struct inode *, struct dentry *,
19     umode_t, dev_t);
20     int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,
21     struct inode *, struct dentry *, unsigned int);
22     int (*setattr) (struct mnt_idmap *, struct dentry *, struct iattr *);
23     int (*getattr) (struct mnt_idmap *, const struct path *,
24     struct kstat *, u32, unsigned int);
25     ssize_t (*listxattr) (struct dentry *, char *, size_t);
26     int (*fiemap) (struct inode *, struct fiemap_extent_info *, u64 start,
27     u64 len);
28     int (*update_time) (struct inode *, int);
29     int (*atomic_open) (struct inode *, struct dentry *,
30     struct file *, unsigned open_flag,
31     umode_t create_mode);
32     int (*tmpfile) (struct mnt_idmap *, struct inode *,
33     struct file *, umode_t);
34     struct posix_acl * (*get_acl) (struct mnt_idmap *, struct dentry *,
35     int);
36     int (*set_acl) (struct mnt_idmap *, struct dentry *,
37     struct posix_acl *, int);
38     int (*fileattr_set) (struct mnt_idmap *idmap,
39     struct dentry *dentry, struct fileattr *fa);
40     int (*fileattr_get) (struct dentry *dentry, struct fileattr *fa);
41     struct offset_ctx * (*get_offset_ctx) (struct inode *inode);
42 } ____cacheline_aligned;

```



```

1 struct file_system_type {
2     const char *name;
3     int fs_flags;
4     #define FS_REQUIRES_DEV      1
5     #define FS_BINARY_MOUNTDATA  2
6     #define FS_HAS_SUBTYPE       4
7     #define FS_USERNS_MOUNT      8      /* Can be mounted by usersns root */
8     #define FS_DISALLOW_NOTIFY_PERM 16   /* Disable fanotify permission events */
9     #define FS_ALLOW_IDMAP       32      /* FS has been updated to handle vfs
idmappings. */
10    #define FS_RENAME_DOES_D_MOVE 32768  /* FS will handle d_move() during rename()
internally. */
11    int (*init_fs_context)(struct fs_context *);
12    const struct fs_parameter_spec *parameters;
13    struct dentry *(*mount)(struct file_system_type *, int,
14    const char *, void *);
15    void (*kill_sb)(struct super_block *);
16    struct module *owner;
17    struct file_system_type *next;
18    struct hlist_head fs_supers;
19
20    struct lock_class_key s_lock_key;
21    struct lock_class_key s_umount_key;
22    struct lock_class_key s_vfs_rename_key;
23    struct lock_class_key s_writers_key[SB_FREEZE_LEVELS];
24
25    struct lock_class_key i_lock_key;
26    struct lock_class_key i_mutex_key;
27    struct lock_class_key invalidate_lock_key;
28    struct lock_class_key i_mutex_dir_key;
29 };

```

```

1 struct fs_struct {
2     int users;
3     spinlock_t lock;
4     seqcount_spinlock_t seq;
5     int umask;
6     int in_exec;
7     struct path root, pwd;
8 } __randomize_layout;

```

```

1 struct path {
2     struct vfsmount *mnt;
3     struct dentry *dentry;
4 } __randomize_layout;

```

```

1 struct files_struct {
2     /*
3     * read mostly part
4     */
5     atomic_t count;
6     bool resize_in_progress;
7     wait_queue_head_t resize_wait;
8     struct fdtable __rcu *fdt;
9     struct fdtable fdtab;
10    /*
11    * written part on a separate cache line in SMP
12    */
13    spinlock_t file_lock ____cacheline_aligned_in_smp;
14    unsigned int next_fd;
15    unsigned long close_on_exec_init[1];
16    unsigned long open_fds_init[1];
17    unsigned long full_fds_bits_init[1];
18    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
19 };

```



```

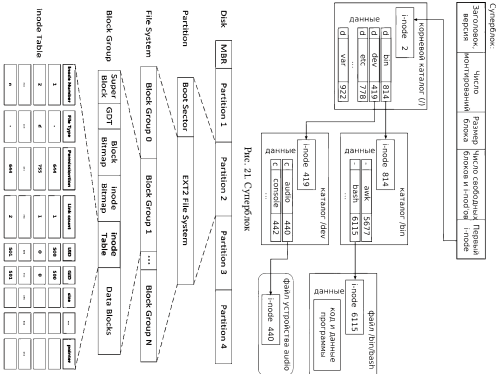
1 struct file {
2     union {
3         /* fput() uses task work when closing and freeing file (default). */
4         struct callback_head    f_task_work;
5         /* fput() must use workqueue (most kernel threads). */
6         struct llist_node       f_llist;
7         unsigned int            f_iocb_flags;
8     };
9     /* Protects f_ep, f_flags. Must not be taken from IRQ context. */
10    spinlock_t                   f_lock;
11    fmode_t                      f_mode;
12    atomic_long_t                f_count;
13    struct mutex                 f_pos_lock;
14    loff_t                      f_pos;
15    unsigned int                 f_flags;
16    struct fown_struct           f_owner;
17    const struct cred            *f_cred;
18    struct file_ra_state         f_ra;
19    struct path                  f_path;
20    struct inode                 *f_inode;          /* cached value */
21    const struct file_operations *f_op;
22    u64                          f_version;
23    #ifdef CONFIG_SECURITY
24    void                         *f_security;
25    #endif
26    /* needed for tty driver, and maybe others */
27    void                         *private_data;
28    #ifdef CONFIG_EPOLL
29    /* Used by fs/eventpoll.c to link all the hooks to this file */
30    struct hlist_head            *f_ep;
31    #endif /* #ifdef CONFIG_EPOLL */
32    struct address_space         *f_mapping;
33    errseq_t                     f_wb_err;
34    errseq_t                     f_sb_err; /* for syncfs */
35 } __randomize_layout
36 __attribute__((aligned(4)));

```

```

1 struct _IO_FILE
2 {
3     int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
4     /* The following pointers correspond to the C++ streambuf protocol. */
5     char *_IO_read_ptr; /* Current read pointer */
6     char *_IO_read_end; /* End of get area. */
7     char *_IO_read_base; /* Start of putback+get area. */
8     char *_IO_write_base; /* Start of put area. */
9     char *_IO_write_ptr; /* Current put pointer. */
10    char *_IO_write_end; /* End of put area. */
11    char *_IO_buf_base; /* Start of reserve area. */
12    char *_IO_buf_end; /* End of reserve area. */
13    /* The following fields are used to support backing up and undo. */
14    char *_IO_save_base; /* Pointer to start of non-current get area. */
15    char *_IO_backup_base; /* Pointer to first valid character of backup area */
16    char *_IO_save_end; /* Pointer to end of non-current get area. */
17    struct _IO_marker *_markers;
18
19    struct _IO_FILE *_chain;
20    int _fileno;
21    int _flags2;
22    __off_t _old_offset; /* This used to be _offset but it's too small. */
23    /* 1+column number of pbase(); 0 is unknown. */
24    unsigned short _cur_column;
25    signed char _vtable_offset;
26    char _shortbuf[1];
27    _IO_lock_t *_lock;
28    #ifdef _IO_USE_OLD_IO_FILE
29 };

```



101 / 287

[illegible]

1	if (isFinite (value)) node.b.b =
2	value; else node.b.b = 0;
3	return node;
4	function _clone (value) {
5	if (value === null value ===
6	undefined) return value;
7	if (value instanceof Date) {
8	return new Date (value.getTime());
9	if (value instanceof RegExp) {
10	return new RegExp (value.source,
11	value.flags);
12	if (value instanceof Function) {
13	return function () {
14	return value.apply (this, arguments);
15	};
16	if (value instanceof Array) {
17	var len = value.length;
18	var res = new Array (len);
19	for (var i = 0; i < len; i++) {
20	res[i] = _clone (value[i]);
21	return res;
22	if (value instanceof Object) {
23	var keys = Object.keys (value);
24	var res = {};
25	for (var i = 0; i < keys.length; i++) {
26	res[keys[i]] = _clone (value[keys[i]]);
27	return res;
28	if (value instanceof String) {
29	return value;
30	if (value instanceof Boolean) {
31	return value;
32	if (value instanceof Number) {
33	return value;

106 / 287

6.7 Структура `super_operations`

Для каждой фс определяется свой набор операций над объектами суперблока. Поскольку разработчики Linux ставят задачу обеспечить интерфейс для большинства фс, перечислять операции на объектах суперблока широко:

```

1  extract super_block {
2    let (super_block, super_block_val) :=
3    read (memory_block (extract super_block_val))
4    read (free_block (extract super_block_val))
5    read (free_block (extract super_block_val))
6    read (free_block (extract super_block_val))
7    read (free_block (extract super_block_val))
8    ...
9    read (free_block (extract super_block_val))
10   }
11   let (super_block, super_block_val) :=
12   read (memory_block (extract super_block_val))
13   read (memory_block (extract super_block_val))
14   let (super_block, super_block_val) :=
15   read (memory_block (extract super_block_val))
16   let (super_block, super_block_val) :=
17   read (memory_block (extract super_block_val))
18   ...
19   read (memory_block (extract super_block_val))
20   }

```

[illegible]

— `union`, `begin` — выделенный VFS для управления мониторингом; использовать отсюда `fs`, например `NIS` (`Network file System`).

В первом `super_operations` нет функции создания суперблока как `struct_operations` создает отформатированный `alloc_super` (he doesn't in `struct super_operations` создавать выделенный и инициализировать новую структуру `super_block` и возвращать ее на нее).

[illegible]

Когда ФС монтируется, создается структура `struct vfstoolt`, которая представляет конкретный экземпляр смонтированной ФС:

```

1 struct vrfmount {
2     struct dentry *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb; /* pointer to superblock */
4     int mnt_flags;
5     struct mnt_idmap *mnt_idmap;
6 };

```

Листинг 30. Структура `vi` имени

103 / 287

[illegible]

Ключи: - $t < \text{тип } \Phi C$ - 0 <лицы ΦC >, то есть полная форма

Товар маркировки — любой материал в самой фс, нужно создать копию или скан (фото) — именно он маркируется. Ключ — использовать: сканера, сама можете отсканировать тип фс. После маркировки материал и фидная фс, сразу типа. Объект маркировки, прежде и при использовании.

С помощью кода маркировки [1-1] [1-2] [1-3] можно получить список клиентов, анализ фидовых счетов типа, фидомаркировка фс, кода маркировки, «свое» не прояснить.

```

1 extern struct density *new_dens(struct file_system *fs, unsigned int flags);
2
3 context char *new_name, void *data, unsigned int flags);
4
5 (struct super_block *, void *, int);
6
7 extern struct density *new_dens(struct file_system *fs, unsigned int flags,
8 void *data, int (*fill_data),
9 (struct super_block *, void *, int));

```

Параметры функций мониторинга:

- `fs_type` – тип файловой системы;

[illegible]

104 / 287

Код для фьючерной системы откомпилируется, но должно быть удалено – эти значения для функции `kill_sb` из `file_system_type`. Выход для этого определены функции

Общую работу по удалению суперблока выполняет генератор `general_shutdown_super`.

Чтобы создать root, надо проинициализировать `inode`, выделив для него память функцией `new_inode()` (внутри не виден аллокатор — `inode` находится в `main` файле `fs.c`, в ней нужно создать корневой каталог (`root`) — `struct super_block` `root`;

```

1 struct Node *new_node(struct t_block *b) {
2     struct Node *node = new_node(b); // allocate node(b);
3     if (node)
4         node->lb_list_add(1, node);
5     return node;
6 }

```

Из соотношения $\text{inode} = \text{filename} \& \text{mask}$, root является корневой директ, которая записывается в поле root структуры. Так точка монтирования (оригинал каталога) оказывается связана с корнем inode .

[illegible]

105 / 287

```

1 struct proc_dir_entry {
2     /*
3      * number of callers into module in progress;
4      * negative -> it's going away RSN
5      */
6     atomic_t in_use;
7     refcount_t refcnt;
8     struct list_head pde_openers; /* who did ->open, but not ->release */
9     /* protects ->pde_openers and all struct pde_opener instances */
10    spinlock_t pde_unload_lock;
11    struct completion *pde_unload_completion;
12    const struct inode_operations *proc_iops;
13    union {
14        const struct proc_ops *proc_ops;
15        const struct file_operations *proc_dir_ops;
16    };
17    const struct dentry_operations *proc_dops;
18    union {
19        const struct seq_operations *seq_ops;
20        int (*single_show)(struct seq_file *, void *);
21    };
22    proc_write_t write;
23    void *data;
24    unsigned int state_size;
25    unsigned int low_ino;
26    nlink_t nlink;
27    kuid_t uid;
28    kgid_t gid;
29    loff_t size;
30    struct proc_dir_entry *parent;
31    struct rb_root subdir;
32    struct rb_node subdir_node;
33    char *name;
34    umode_t mode;
35    u8 flags;
36    u8 namelen;
37    char inline_name[];
38 } __randomize_layout;

```

```

1 struct proc_ops {
2     unsigned int proc_flags;
3     int (*proc_open)(struct inode *, struct file *);
4     ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
6     ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
7     /* mandatory unless nonseekable_open() or equivalent is used */
8     loff_t (*proc_lseek)(struct file *, loff_t, int);
9     int (*proc_release)(struct inode *, struct file *);
10    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
11    long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
12    #ifdef CONFIG_COMPAT
13    long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
14    #endif
15    int (*proc_mmap)(struct file *, struct vm_area_struct *);
16    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long, unsigned long,
17    unsigned long, unsigned long);
18 } __randomize_layout;

```



```

1 struct file_operations {
2     struct module *owner;
3     loff_t (*llseek) (struct file *, loff_t, int);
4     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
6     ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
7     ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
8     int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
9     unsigned int flags);
10    int (*iterate_shared) (struct file *, struct dir_context *);
11    __poll_t (*poll) (struct file *, struct poll_table_struct *);
12    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
13    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
14    int (*mmap) (struct file *, struct vm_area_struct *);
15    unsigned long mmap_supported_flags;
16    int (*open) (struct inode *, struct file *);
17    int (*flush) (struct file *, fl_owner_t id);
18    int (*release) (struct inode *, struct file *);
19    int (*fsync) (struct file *, loff_t, loff_t, int datasync);
20    int (*fasync) (int, struct file *, int);
21    int (*lock) (struct file *, int, struct file_lock *);
22    unsigned long (*get_unmapped_area)(struct file *, unsigned long, unsigned long,
23    unsigned long, unsigned long);
24    int (*check_flags)(int);
25    int (*flock) (struct file *, int, struct file_lock *);
26    ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t *, size_t,
27    unsigned int);
28    ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info *, size_t,
29    unsigned int);
30    void (*splice_eof)(struct file *file);
31    int (*setlease)(struct file *, int, struct file_lock **, void **);
32    long (*fallocate)(struct file *file, int mode, loff_t offset,
33    loff_t len);
34    void (*show_fdinfo)(struct seq_file *m, struct file *f);
35    #ifndef CONFIG_MMU
36    unsigned (*mmap_capabilities)(struct file *);
37    #endif
38    ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
39    loff_t, size_t, unsigned int);
40    loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
41    struct file *file_out, loff_t pos_out,
42    loff_t len, unsigned int remap_flags);
43    int (*fadvise)(struct file *, loff_t, loff_t, int);
44    int (*uring_cmd)(struct io_uring_cmd *ioucmd, unsigned int issue_flags);
45    int (*uring_cmd_iopoll)(struct io_uring_cmd *, struct io_comp_batch *,
46    unsigned int poll_flags);
47 } __randomize_layout;

```



```

1 struct seq_file {
2     char *buf;
3     size_t size;
4     size_t from;
5     size_t count;
6     size_t pad_until;
7     loff_t index;
8     loff_t read_pos;
9     struct mutex lock;
10    const struct seq_operations *op;
11    int poll_event;
12    const struct file *file;
13    void *private;
14 };

```

```

1 struct seq_operations {
2     void * (*start) (struct seq_file *m, loff_t *pos);
3     void (*stop) (struct seq_file *m, void *v);
4     void * (*next) (struct seq_file *m, void *v, loff_t *pos);
5     int (*show) (struct seq_file *m, void *v);
6 };

```

```

1 int single_open(struct file *file, int (*show)(struct seq_file *, void *),
2 void *data)
3 {
4     struct seq_operations *op = kmalloc(sizeof(*op), GFP_KERNEL_ACCOUNT);
5     int res = -ENOMEM;
6
7     if (op) {
8         op->start = single_start;
9         op->next = single_next;
10        op->stop = single_stop;
11        op->show = show;
12        res = seq_open(file, op);
13        if (!res)
14            ((struct seq_file *)file->private_data)->private = data;
15        else
16            kfree(op);
17    }
18    return res;
19 }
20 EXPORT_SYMBOL(single_open);

```

```

1 #include <arpa/inet.h>
2
3 int main() {
4     struct sockaddr_in server_addr;
5     struct sockaddr_in client_addr;
6     int sockfd;
7     int portno = 8080;
8     int len = sizeof(int);
9     int val;
10
11     sockfd = socket(AF_INET, SOCK_STREAM, 0);
12     if (sockfd < 0) {
13         perror("socket failed");
14         return 1;
15     }
16
17     server_addr.sin_family = AF_INET;
18     server_addr.sin_port = htons(portno);
19     inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);
20
21     if (connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
22         perror("connect failed");
23         return 1;
24     }
25
26     val = 42;
27     send(sockfd, &val, len, 0);
28
29     return 0;
30 }

```

[illegible]

8.7 Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексирования (таб. раб.)

8.7.1 Сепараторы

Решается классическая задача проинкоде-тирования. У нас 2 типа клиентов – клиент-производитель и клиент-потребитель. Клиент посылает серверу пропсы, как оп P (производитель) или C (потребитель, потребитель), то есть что он хочет сделать – положить данные в какой-то буфер или считать данные из буфера.

Сравнение мультимедийных и одноязычных событий, дальнейшее продвижение к нему, обеспечивалось соединением (по схеме из предыдущего раздела) кода мы переводили с одного на другой, проведя дублирование. Это был последний шаг — к нему мы возвращались, (практически как у потока в многопоточной версии).

Синхронизация

— Не нужно беспокоиться, малоиспользование (в-продолжительности, безразлично) а симпатизация куклы сглаживающие сенсорию интуитивный не почувствует, а интуитивный будет переключен. Как опустить, что будет запущено? — Все равно вводить сетевую.

— В обеих версиях (многопоточной и с мультиплексированием) движки быть сенсорию, но в мультиплексоре не должно быть интуитивного.

```

135 }
136 close(fd);
137 if (semctl(semaphore_id, 0, IPC_RMID) == -1) {
138     perror("semctl");
139     exit(1);
140 }
141 return EXIT_SUCCESS;
142 }

```

```

1 # include <stdio.h>
2 # include <stdlib.h>
3 # include <unistd.h>
4 # include <string.h>
5 # include <sys/types.h>
6 # include <sys/stat.h>
7 # include <fcntl.h>
8
9 #define OPEN_FLAGS O_RDONLY
10 #define OPEN_FLAGS O_RDWR
11 #define MODE_S128 0
12 #define MODE_S128 8
13 #define SECTOR_SIZE, 512
14 #define SECTOR_SIZE, 1024
15
16 #define SECTOR_SIZE, 1024
17
18 #define SECTOR_SIZE, 1024
19
20 #define SECTOR_SIZE, 1024
21
22 #define SECTOR_SIZE, 1024
23
24 #define SECTOR_SIZE, 1024
25
26 #define SECTOR_SIZE, 1024
27
28 #define SECTOR_SIZE, 1024
29
30 #define SECTOR_SIZE, 1024
31
32 #define SECTOR_SIZE, 1024
33
34 #define SECTOR_SIZE, 1024
35
36 #define SECTOR_SIZE, 1024
37
38 #define SECTOR_SIZE, 1024
39
40 #define SECTOR_SIZE, 1024
41
42 #define SECTOR_SIZE, 1024
43
44 #define SECTOR_SIZE, 1024
45
46 #define SECTOR_SIZE, 1024
47
48 #define SECTOR_SIZE, 1024
49
50 #define SECTOR_SIZE, 1024
51
52 #define SECTOR_SIZE, 1024
53
54 #define SECTOR_SIZE, 1024
55
56 #define SECTOR_SIZE, 1024
57
58 #define SECTOR_SIZE, 1024
59
60 #define SECTOR_SIZE, 1024
61
62 #define SECTOR_SIZE, 1024
63
64 #define SECTOR_SIZE, 1024
65
66 #define SECTOR_SIZE, 1024
67
68 #define SECTOR_SIZE, 1024
69
70 #define SECTOR_SIZE, 1024
71
72 #define SECTOR_SIZE, 1024
73
74 #define SECTOR_SIZE, 1024
75
76 #define SECTOR_SIZE, 1024
77
78 #define SECTOR_SIZE, 1024
79
80 #define SECTOR_SIZE, 1024
81
82 #define SECTOR_SIZE, 1024
83
84 #define SECTOR_SIZE, 1024
85
86 #define SECTOR_SIZE, 1024
87
88 #define SECTOR_SIZE, 1024
89
90 #define SECTOR_SIZE, 1024
91
92 #define SECTOR_SIZE, 1024
93
94 #define SECTOR_SIZE, 1024
95
96 #define SECTOR_SIZE, 1024
97
98 #define SECTOR_SIZE, 1024
99
100 #define SECTOR_SIZE, 1024

```

— Читателям это кажется, что вольнодумцы перешли в ряды атеистов, но это было бы заблуждением. Считается, что атеизм — это отрицание существования бога, но это не совсем так. Атеизм — это отрицание некоего высшего существа, которое не имеет ни пола, ни возраста, ни национальности, ни происхождения, ни смерти. Атеизм — это отрицание некоего высшего существа, которое не имеет ни пола, ни возраста, ни национальности, ни происхождения, ни смерти. Атеизм — это отрицание некоего высшего существа, которое не имеет ни пола, ни возраста, ни национальности, ни происхождения, ни смерти.

ответ не получен. Клиент блокирован на запросе receive from, и только после получения он сможет выполнить следующую итерацию.

— Clarus

В одностороннем сердце с мультисистемным блоком в суставе отстоять бы весь сервер, включая обслуживающие персонал, которые могли бы обслуживать место в бункере — получить бы дебрид. Поэтому проверку скарпана в такой версии нужно делать неосторожно (напрямую сел в гробик) и при запуске должно быть отмечено, что нужно попортить память, о чем говорится в комментарии (сказать, что он должен быть?).

1	medium	solid	200
2	# include $\sqrt{2}$		
3	# include $\sqrt{2}$		
4	# include $\sqrt{2}$		
5	# include $\sqrt{2}$		
6	# include $\sqrt{2}$		
7	# include $\sqrt{2}$		
8	# include $\sqrt{2}$		
9	# include $\sqrt{2}$		
10	# include $\sqrt{2}$		
11	# include $\sqrt{2}$		
12	# include $\sqrt{2}$		
13	# include $\sqrt{2}$		
14	# include $\sqrt{2}$		
15	# include $\sqrt{2}$		
16	# include $\sqrt{2}$		
17	# include $\sqrt{2}$		
18	# include $\sqrt{2}$		
19	# include $\sqrt{2}$		
20	# include $\sqrt{2}$		
21	# include $\sqrt{2}$		
22	# include $\sqrt{2}$		

32	return false;	32	return false;
33		33	if (this instanceof Object) {
34	clear responses(0, SIZE) = 0;	34	clear responses(0, SIZE) = 0;
35		35	if (this instanceof Response) {
36	if (this instanceof Response) {	36	clear responses(0, SIZE) = 0;
37	return true;	37	return true;
38		38	if (this instanceof Response) {
39	clear responses(0, SIZE) = 0;	39	clear responses(0, SIZE) = 0;
40	return true;	40	return true;
41		41	clear responses(0, SIZE) = 0;
42		42	clear responses(0, SIZE) = 0;
43		43	clear responses(0, SIZE) = 0;
44		44	clear responses(0, SIZE) = 0;
45		45	clear responses(0, SIZE) = 0;
46		46	clear responses(0, SIZE) = 0;
47		47	clear responses(0, SIZE) = 0;
48		48	clear responses(0, SIZE) = 0;
49		49	clear responses(0, SIZE) = 0;
50		50	clear responses(0, SIZE) = 0;
51		51	clear responses(0, SIZE) = 0;
52		52	clear responses(0, SIZE) = 0;
53		53	clear responses(0, SIZE) = 0;
54		54	clear responses(0, SIZE) = 0;
55		55	clear responses(0, SIZE) = 0;
56		56	clear responses(0, SIZE) = 0;
57		57	clear responses(0, SIZE) = 0;
58		58	clear responses(0, SIZE) = 0;
59		59	clear responses(0, SIZE) = 0;
60		60	clear responses(0, SIZE) = 0;
61		61	clear responses(0, SIZE) = 0;
62		62	clear responses(0, SIZE) = 0;
63		63	clear responses(0, SIZE) = 0;
64		64	clear responses(0, SIZE) = 0;
65		65	clear responses(0, SIZE) = 0;
66		66	clear responses(0, SIZE) = 0;
67		67	clear responses(0, SIZE) = 0;
68		68	clear responses(0, SIZE) = 0;
69		69	clear responses(0, SIZE) = 0;
70		70	clear responses(0, SIZE) = 0;
71		71	clear responses(0, SIZE) = 0;
72		72	clear responses(0, SIZE) = 0;
73		73	clear responses(0, SIZE) = 0;
74		74	clear responses(0, SIZE) = 0;
75		75	clear responses(0, SIZE) = 0;
76		76	clear responses(0, SIZE) = 0;
77		77	clear responses(0, SIZE) = 0;
78		78	clear responses(0, SIZE) = 0;
79		79	clear responses(0, SIZE) = 0;
80		80	clear responses(0, SIZE) = 0;
81		81	clear responses(0, SIZE) = 0;
82		82	clear responses(0, SIZE) = 0;
83		83	clear responses(0, SIZE) = 0;
84		84	clear responses(0, SIZE) = 0;
85		85	clear responses(0, SIZE) = 0;
86		86	clear responses(0, SIZE) = 0;
87		87	clear responses(0, SIZE) = 0;
88		88	clear responses(0, SIZE) = 0;
89		89	clear responses(0, SIZE) = 0;
90		90	clear responses(0, SIZE) = 0;
91		91	clear responses(0, SIZE) = 0;
92		92	clear responses(0, SIZE) = 0;
93		93	clear responses(0, SIZE) = 0;
94		94	clear responses(0, SIZE) = 0;
95		95	clear responses(0, SIZE) = 0;
96		96	clear responses(0, SIZE) = 0;
97		97	clear responses(0, SIZE) = 0;
98		98	clear responses(0, SIZE) = 0;
99		99	clear responses(0, SIZE) = 0;
100		100	clear responses(0, SIZE) = 0;

[illegible]

```

76         return EXIT_FAILURE;
77     }
78     rc = consume(td);
79     }
80     close(fd);
81     if (rc == -1) return EXIT_FAILURE;
82     }
83     return EXIT_SUCCESS;
84 }

```

```

637 read_sids = (read_sids - 1) % NB_SIDS
638
639 smoothes[smoothes_id, last_read_write, 1:]
640         = read_sids[read_sids_id, last_read_write, 1:]
641         + (read_sids - 1) % NB_SIDS
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
15
```

1311	perfor("accept");
1312	exit();
1313	if (strcmp(temperature, "SIN", "TWO", "ALL", "OTHER") == 1) {
1314	perfor("error");
1315	exit();
1316	if (1) {
1317	pool["fe" = pool["center100];
1318	if (pool["fe" == 1) {
1319	perfor("pool", "center10");
1320	exit();
1321	if (1) {
1322	perfor("pool", "center10");
1323	perfor("pool", "center10");
1324	if (pool["center10"] == 1) {
1325	perfor("pool", "center10");
1326	exit();
1327	perfor("pool", "center10");
1328	exit();
1329	perfor("pool", "center10");
1330	exit();
1331	perfor("pool", "center10");
1332	exit();
1333	perfor("pool", "center10");
1334	exit();
1335	perfor("pool", "center10");
1336	exit();
1337	perfor("pool", "center10");
1338	exit();
1339	perfor("pool", "center10");
1340	exit();
1341	perfor("pool", "center10");
1342	exit();
1343	perfor("pool", "center10");
1344	exit();
1345	perfor("pool", "center10");
1346	exit();
1347	perfor("pool", "center10");
1348	exit();
1349	perfor("pool", "center10");
1350	exit();
1351	perfor("pool", "center10");
1352	exit();
1353	perfor("pool", "center10");
1354	exit();
1355	perfor("pool", "center10");
1356	exit();
1357	perfor("pool", "center10");
1358	exit();
1359	perfor("pool", "center10");
1360	exit();
1361	perfor("pool", "center10");
1362	exit();
1363	perfor("pool", "center10");
1364	exit();
1365	perfor("pool", "center10");
1366	exit();
1367	perfor("pool", "center10");
1368	exit();
1369	perfor("pool", "center10");
1370	exit();
1371	perfor("pool", "center10");
1372	exit();
1373	perfor("pool", "center10");
1374	exit();
1375	perfor("pool", "center10");
1376	exit();
1377	perfor("pool", "center10");
1378	exit();
1379	perfor("pool", "center10");
1380	exit();
1381	perfor("pool", "center10");
1382	exit();
1383	perfor("pool", "center10");
1384	exit();
1385	perfor("pool", "center10");
1386	exit();
1387	perfor("pool", "center10");
1388	exit();
1389	perfor("pool", "center10");
1390	exit();
1391	perfor("pool", "center10");
1392	exit();
1393	perfor("pool", "center10");
1394	exit();
1395	perfor("pool", "center10");
1396	exit();
1397	perfor("pool", "center10");
1398	exit();
1399	perfor("pool", "center10");
1400	exit();
1401	perfor("pool", "center10");
1402	exit();
1403	perfor("pool", "center10");
1404	exit();
1405	perfor("pool", "center10");
1406	exit();
1407	perfor("pool", "center10");
1408	exit();
1409	perfor("pool", "center10");
1410	exit();
1411	perfor("pool", "center10");
1412	exit();
1413	perfor("pool", "center10");
1414	exit();
1415	perfor("pool", "center10");
1416	exit();
1417	perfor("pool", "center10");
1418	exit();
1419	perfor("pool", "center10");
1420	exit();
1421	perfor("pool", "center10");
1422	exit();
1423	perfor("pool", "center10");
1424	exit();
1425	perfor("pool", "center10");
1426	exit();
1427	perfor("pool", "center10");
1428	exit();
1429	perfor("pool", "center10");
1430	exit();
1431	perfor("pool", "center10");
1432	exit();
1433	perfor("pool", "center10");
1434	exit();
1435	perfor("pool", "center10");
1436	exit();
1437	perfor("pool", "center10");
1438	exit();
1439	perfor("pool", "center10");
1440	exit();
1441	perfor("pool", "center10");
1442	exit();
1443	perfor("pool", "center10");
1444	exit();
1445	perfor("pool", "center10");
1446	exit();
1447	perfor("pool", "center10");
1448	exit();
1449	perfor("pool", "center10");
1450	exit();
1451	perfor("pool", "center10");
1452	exit();
1453	perfor("pool", "center10");
1454	exit();
1455	perfor("pool", "center10");
1456	exit();
1457	perfor("pool", "center10");
1458	exit();
1459	perfor("pool", "center10");
1460	exit();
1461	perfor("pool", "center10");
1462	exit();
1463	perfor("pool", "center10");
1464	exit();
1465	perfor("pool", "center10");
1466	exit();
1467	perfor("pool", "center10");
1468	exit();
1469	perfor("pool", "center10");
1470	exit();
147	

```

1 struct kmem_cache {
2     #ifndef CONFIG_SLUB_TINY
3     struct kmem_cache_cpu __percpu *cpu_slab;
4     #endif
5     /* Used for retrieving partial slabs, etc. */
6     slab_flags_t flags;
7     unsigned long min_partial;
8     unsigned int size; /* Object size including metadata */
9     unsigned int object_size; /* Object size without metadata */
10    struct reciprocal_value reciprocal_size;
11    unsigned int offset; /* Free pointer offset */
12    #ifdef CONFIG_SLUB_CPU_PARTIAL
13    /* Number of per cpu partial objects to keep around */
14    unsigned int cpu_partial;
15    /* Number of per cpu partial slabs to keep around */
16    unsigned int cpu_partial_slabs;
17    #endif
18    struct kmem_cache_order_objects oo; /* Allocation and freeing of slabs */
19    struct kmem_cache_order_objects min;
20    gfp_t allocflags; /* gfp flags to use on each alloc */
21    int refcount; /* Refcount for slab cache destroy */
22    void (*ctor)(void *object); /* Object constructor */
23    unsigned int inuse; /* Offset to metadata */
24    unsigned int align; /* Alignment */
25    unsigned int red_left_pad; /* Left redzone padding size */
26    const char *name; /* Name (only for display!) */
27    struct list_head list; /* List of slab caches */
28    #ifdef CONFIG_SYSFS
29    struct kobject kobj; /* For sysfs */
30    #endif
31    #ifdef CONFIG_SLAB_FREELIST_HARDENED
32    unsigned long random;
33    #endif
34    #ifdef CONFIG_NUMA
35    /** Defragmentation by allocating from a remote node. */
36    unsigned int remote_node_defrag_ratio;
37    #endif
38    #ifdef CONFIG_SLAB_FREELIST_RANDOM
39    unsigned int *random_seq;
40    #endif
41    #ifdef CONFIG_KASAN_GENERIC
42    struct kasan_cache kasan_info;
43    #endif
44    #ifdef CONFIG_HARDENED_USERCOPY
45    unsigned int useroffset; /* Usercopy region offset */
46    unsigned int usersize; /* Usercopy region size */
47    #endif
48    struct kmem_cache_node *node[MAX_NUMNODES];
49 };

```



```

1 struct irqaction {
2     irq_handler_t      handler;
3     void              *dev_id;
4     void __percpu      *percpu_dev_id;
5     struct irqaction   *next;
6     irq_handler_t      thread_fn;
7     struct task_struct *thread;
8     struct irqaction   *secondary;
9     unsigned int       irq;
10    unsigned int        flags;
11    unsigned long        thread_flags;
12    unsigned long        thread_mask;
13    const char           *name;
14    struct proc_dir_entry *dir;
15 } ____cacheline_internodealigned_in_smp;

```

```

1 struct softirq_action {
2     void (*action)(struct softirq_action *);
3 };

```

```

1 static struct softirq_action softirq_vec[NR_SOFTIRQS] __cacheline_aligned_in_smp;
2 const char * const softirq_to_name[NR_SOFTIRQS] = {
3     "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL",
4     "TASKLET", "SCHED", "HRTIMER", "RCU"
5 };

```

```

1 struct tasklet_struct
2 {
3     struct tasklet_struct *next;
4     unsigned long state;
5     atomic_t count;
6     bool use_callback;
7     union {
8         void (*func)(unsigned long data);
9         void (*callback)(struct tasklet_struct *t);
10    };
11    unsigned long data;
12 };

```



```

1 struct workqueue_struct {
2     struct list_head    pwqs;           /* WR: all pwqs of this wq */
3     struct list_head    list;           /* PR: list of all workqueues */
4
5     struct mutex         mutex;          /* protects this wq */
6     int                 work_color;      /* WQ: current work color */
7     int                 flush_color;     /* WQ: current flush color */
8     atomic_t            nr_pwqs_to_flush; /* flush in progress */
9     struct wq_flusher    *first_flusher; /* WQ: first flusher */
10    struct list_head     flusher_queue;  /* WQ: flush waiters */
11    struct list_head     flusher_overflow; /* WQ: flush overflow list */
12
13    struct list_head     maydays;        /* MD: pwqs requesting rescue */
14    struct worker         *rescuer;        /* MD: rescue worker */
15
16    int                 nr_drainers;      /* WQ: drain in progress */
17    int                 saved_max_active; /* WQ: saved pwq max_active */
18
19    struct workqueue_attrs *unbound_attrs; /* PW: only for unbound wqs */
20    struct pool_workqueue *dfl_pwq;       /* PW: only for unbound wqs */
21
22    #ifdef CONFIG_SYSFS
23    struct wq_device      *wq_dev;        /* I: for sysfs interface */
24    #endif
25    #ifdef CONFIG_LOCKDEP
26    char                  *lock_name;
27    struct lock_class_key key;
28    struct lockdep_map    lockdep_map;
29    #endif
30    char                  name[WQ_NAME_LEN]; /* I: workqueue name */
31
32    /*
33     * Destruction of workqueue_struct is RCU protected to allow walking
34     * the workqueues list without grabbing wq_pool_mutex.
35     * This is used to dump all workqueues from sysrq.
36     */
37    struct rcu_head        rcu;
38
39    /* hot fields used during command issue, aligned to cacheline */
40    unsigned int          flags ____cacheline_aligned; /* WQ: WQ_* flags */
41    struct pool_workqueue __percpu __rcu **cpu_pwq; /* I: per-cpu pwqs */
42 };

```

```

1 struct work_struct {
2     atomic_long_t data;
3     struct list_head entry;
4     work_func_t func;
5     #ifdef CONFIG_LOCKDEP
6     struct lockdep_map lockdep_map;
7     #endif
8 };

```



```

1 struct device {
2     struct kobject kobj;
3     struct device      *parent;
4     struct device_private *p;
5     const char          *init_name; /* initial name of the device */
6     const struct device_type *type;
7     const struct bus_type *bus; /* type of bus device is on */
8     struct device_driver *driver; /* which driver has allocated this device */
9     void *platform_data; /* Platform specific data, device
10    core doesn't touch it */
11    void *driver_data; /* Driver data, set and get with
12    dev_set_drvdata/dev_get_drvdata */
13    struct mutex mutex; /* mutex to synchronize calls to * its driver. */
14    struct dev_links_info links;
15    struct dev_pm_info power;
16    struct dev_pm_domain *pm_domain;
17    #ifdef CONFIG_ENERGY_MODEL
18    struct em_perf_domain *em_pd;
19    #endif
20    #ifdef CONFIG_PINCTRL
21    struct dev_pin_info *pins;
22    #endif
23    struct dev_msi_info msi;
24    #ifdef CONFIG_DMA_OPS
25    const struct dma_map_ops *dma_ops;
26    #endif
27    u64 *dma_mask; /* dma mask (if dma'able device) */
28    u64 coherent_dma_mask; /* Like dma_mask, but for
29    alloc_coherent mappings as
30    not all hardware supports
31    64 bit addresses for consistent
32    allocations such descriptors. */
33    u64 bus_dma_limit; /* upstream dma constraint */
34    const struct bus_dma_region *dma_range_map;
35    struct device_dma_parameters *dma_parms;
36    struct list_head dma_pools; /* dma pools (if dma'ble) */
37    #ifdef CONFIG_DMA_DECLARE_COHERENT
38    struct dma_coherent_mem *dma_mem; /* internal for coherent mem override */
39    #endif
40    #ifdef CONFIG_DMA_CMA
41    struct cma *cma_area; /* contiguous memory area for dma allocations */
42    #endif
43    #ifdef CONFIG_SWIOTLB
44    struct io_tlb_mem *dma_io_tlb_mem;
45    #endif
46    #ifdef CONFIG_SWIOTLB_DYNAMIC
47    struct list_head dma_io_tlb_pools;
48    spinlock_t dma_io_tlb_lock;
49    bool dma_uses_io_tlb;
50    #endif
51    struct dev_archdata archdata; /* arch specific additions */
52    struct device_node *of_node; /* associated device tree node */
53    struct fwnode_handle *fwnode; /* firmware device node */
54    #ifdef CONFIG_NUMA
55    int numa_node; /* NUMA node this device is close to */
56    #endif
57    dev_t devt; /* dev_t, creates the sysfs "dev" */
58    u32 id; /* device instance */
59    spinlock_t devres_lock;
60    struct list_head devres_head;
61    const struct class *class;
62    const struct attribute_group **groups; /* optional groups */
63    void (*release)(struct device *dev);
64    struct iommu_group *iommu_group;
65    struct dev_iommu *iommu;
66    struct device_physical_location *physical_location;
67    enum device_removable removable;
68    bool offline_disabled:1;
69    bool offline:1;
70    bool of_node_reused:1;

```



```

71     bool                state_synced:1;
72     bool                can_match:1;
73     #if defined(CONFIG_ARCH_HAS_SYNC_DMA_FOR_DEVICE) || \
74     defined(CONFIG_ARCH_HAS_SYNC_DMA_FOR_CPU) || \
75     defined(CONFIG_ARCH_HAS_SYNC_DMA_FOR_CPU_ALL)
76     bool                dma_coherent:1;
77     #endif
78     #ifdef CONFIG_DMA_OPS_BYPASS
79     bool                dma_ops_bypass : 1;
80     #endif
81 };

```

```

1  struct device_driver {
2      const char          *name;
3      const struct bus_type *bus;
4      struct module        *owner;
5      const char          *mod_name;          /* used for built-in modules */
6      bool suppress_bind_attrs;              /* disables bind/unbind via sysfs */
7      enum probe_type probe_type;
8      const struct of_device_id *of_match_table;
9      const struct acpi_device_id *acpi_match_table;
10     int (*probe) (struct device *dev);
11     void (*sync_state) (struct device *dev);
12     int (*remove) (struct device *dev);
13     void (*shutdown) (struct device *dev);
14     int (*suspend) (struct device *dev, pm_message_t state);
15     int (*resume) (struct device *dev);
16     const struct attribute_group **groups;
17     const struct attribute_group **dev_groups;
18     const struct dev_pm_ops *pm;
19     void (*coredump) (struct device *dev);
20     struct driver_private *p;
21 };

```



```

1 struct usb_interface {
2     /* array of alternate settings for this interface,
3      * stored in no particular order */
4     struct usb_host_interface *altsetting;
5
6     struct usb_host_interface *cur_altsetting;    /* the currently
7      * active alternate setting */
8     unsigned num_altsetting;    /* number of alternate settings */
9
10    /* If there is an interface association descriptor then it will list
11     * the associated interfaces */
12    struct usb_interface_assoc_descriptor *intf_assoc;
13
14    int minor;    /* minor number this interface is
15     * bound to */
16    enum usb_interface_condition condition;    /* state of binding */
17    unsigned sysfs_files_created:1; /* the sysfs attributes exist */
18    unsigned ep_devs_created:1;    /* endpoint "devices" exist */
19    unsigned unregistering:1;    /* unregistration is in progress */
20    unsigned needs_remote_wakeup:1; /* driver requires remote wakeup */
21    unsigned needs_altsetting0:1; /* switch to altsetting 0 is pending */
22    unsigned needs_binding:1;    /* needs delayed unbind/rebind */
23    unsigned resetting_device:1; /* true: bandwidth alloc after reset */
24    unsigned authorized:1;    /* used for interface authorization */
25    enum usb_wireless_status wireless_status;
26    struct work_struct wireless_status_work;
27
28    struct device dev;    /* interface specific device info */
29    struct device *usb_dev;
30    struct work_struct reset_ws;    /* for resets in atomic context */
31 };

```

```

1 struct urb {
2     /* private: usb core and host controller only fields in the urb */
3     struct kref kref;    /* reference count of the URB */
4     int unlinked;    /* unlink error code */
5     void *hcpriv;    /* private data for host controller */
6     atomic_t use_count;    /* concurrent submissions counter */
7     atomic_t reject;    /* submissions will fail */
8     /* public: documented fields in the urb that can be used by drivers */
9     struct list_head urb_list;    /* list head for use by the urb's * current owner */
10    struct list_head anchor_list;    /* the URB may be anchored */
11    struct usb_anchor *anchor;
12    struct usb_device *dev;    /* (in) pointer to associated device */
13    struct usb_host_endpoint *ep;    /* (internal) pointer to endpoint */
14    unsigned int pipe;    /* (in) pipe information */
15    unsigned int stream_id;    /* (in) stream ID */
16    int status;    /* (return) non-ISO status */
17    unsigned int transfer_flags;    /* (in) URB_SHORT_NOT_OK | ... */
18    void *transfer_buffer;    /* (in) associated data buffer */
19    dma_addr_t transfer_dma;    /* (in) dma addr for transfer_buffer */
20    struct scatterlist *sg;    /* (in) scatter gather buffer list */
21    int num_mapped_sgs;    /* (internal) mapped sg entries */
22    int num_sgs;    /* (in) number of entries in the sg list */
23    u32 transfer_buffer_length;    /* (in) data buffer length */
24    u32 actual_length;    /* (return) actual transfer length */
25    unsigned char *setup_packet;    /* (in) setup packet (control only) */
26    dma_addr_t setup_dma;    /* (in) dma addr for setup_packet */
27    int start_frame;    /* (modify) start frame (ISO) */
28    int number_of_packets;    /* (in) number of ISO packets */
29    int interval;    /* (modify) transfer interval * (INT/ISO) */
30    int error_count;    /* (return) number of ISO errors */
31    void *context;    /* (in) context for completion */
32    usb_complete_t complete;    /* (in) completion routine */
33    struct usb_iso_packet_descriptor iso_frame_desc[]; /* (in) ISO ONLY */
34 };

```



```

1 struct usb_driver {
2     const char *name;
3
4     int (*probe) (struct usb_interface *intf,
5     const struct usb_device_id *id);
6
7     void (*disconnect) (struct usb_interface *intf);
8
9     int (*unlocked_ioctl) (struct usb_interface *intf, unsigned int code,
10    void *buf);
11
12    int (*suspend) (struct usb_interface *intf, pm_message_t message);
13    int (*resume) (struct usb_interface *intf);
14    int (*reset_resume)(struct usb_interface *intf);
15
16    int (*pre_reset)(struct usb_interface *intf);
17    int (*post_reset)(struct usb_interface *intf);
18
19    const struct usb_device_id *id_table;
20    const struct attribute_group **dev_groups;
21
22    struct usb_dynids dynids;
23    struct device_driver driver;
24    unsigned int no_dynamic_id:1;
25    unsigned int supports_autosuspend:1;
26    unsigned int disable_hub_initiated_lpm:1;
27    unsigned int soft_unbind:1;
28 };

```

```

1 struct usb_device_driver {
2     const char *name;
3
4     bool (*match) (struct usb_device *udev);
5     int (*probe) (struct usb_device *udev);
6     void (*disconnect) (struct usb_device *udev);
7
8     int (*suspend) (struct usb_device *udev, pm_message_t message);
9     int (*resume) (struct usb_device *udev, pm_message_t message);
10
11    int (*choose_configuration) (struct usb_device *udev);
12
13    const struct attribute_group **dev_groups;
14    struct device_driver driver;
15    const struct usb_device_id *id_table;
16    unsigned int supports_autosuspend:1;
17    unsigned int generic_subclass:1;
18 };

```

36	...endif	*/endif	endif
37	...	...	...
38	proc_start = proc_start + 1;	endif	endif
39	...	...	...
40	...	...	...
41	...	...	...
42	static void __init	...	...
43	...	...	...
44	...	...	...
45	...	...	...
46	...	...	...

Листинг 77. Блок ввода модуля «Файловый»

13

13.6. Чертеж структуры VFS – super_block, inode, dentry, file – их назначение

Все выше описанные структуры являются основой взаимодействия и для каждого файла создаются их экземпляры. Вроде бы назначение: super_block описывает суперблок файловой системы, inode – физический файл, dentry – запись в каталоге, file – открытый файл.

13.6.1 super_block – описывает суперблок файловой системы

Суперблок представляет в VFS информацию о суперблоке ФС – структура с данными, необходимыми для контроля и доступа к файлам ФС; начальные данные обобщены в файлах, хранящихся «внутри» (внутренние) соответствующих устройств (таблицы). В файлах ФС-руководств суперблоку соответствуют данные, относящиеся к файлам, хранящимся «снаружи» (внешние) (таблицы).

1	struct super_block {	
2	...;	
3	...;	
4	...;	
5	...;	
6	...;	
7	...;	
8	...;	
9	...;	
10	...;	
11	...;	
12	...;	
13	...;	
14	...;	
15	...;	

Листинг 78. Структура super_block (cont)

– s_blocksize – s_blocksize – размер блока и максимальный размер файла;
– s_magic – магическое число ФС;
– s_type, s_op – тип ФС и функции на суперблоке (super_operations); s_root – точка монтирования (корневой каталог); s_inodes – список всех inode;
– s_mounts – список монтируемых данных ФС (всего их не ФС может быть несколько разных).

Осторожно

Вместо переноса на структуры, которые являются не подклассами (показателями), вероятно, имеет место.

Свойство	Число	Размер	Число	Размер
Свойство	Число	Размер	Число	Размер
Свойство	Число	Размер	Число	Размер

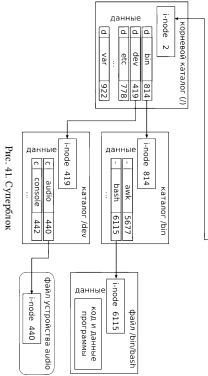


Рис. 41. Структура

13. Файловая система, объединяет файловые подсистемы (link/line, иерархическая структура файловых подсистем). Внутренняя файловая система VFS и link. Внутренняя файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним.

13.2. Особенности файловой подсистемы Unix/Linux: иерархическая структура файловой подсистемы

Файловая система – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним.

13.3. Задачи файловой подсистемы

- 1) Обеспечение доступа к данным;
- 2) Обеспечение доступа к данным;
- 3) Обеспечение доступа к данным;
- 4) Обеспечение доступа к данным;
- 5) Обеспечение доступа к данным;

203 / 287

Осторожно

Вместо переноса на структуры, которые являются не подклассами (показателями), вероятно, имеет место.

File System	Block Group 0	Block Group 1

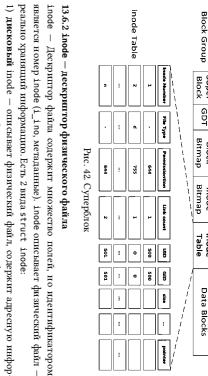


Рис. 42. Структура

– s_blocksize – s_blocksize – размер блока и максимальный размер файла;
– s_magic – магическое число ФС;
– s_type, s_op – тип ФС и функции на суперблоке (super_operations); s_root – точка монтирования (корневой каталог); s_inodes – список всех inode;
– s_mounts – список монтируемых данных ФС (всего их не ФС может быть несколько разных).

208 / 287



Символьный уровень – это уровень, который используется для управления данными и доступа к ним. Базовый уровень – это уровень, который используется для управления данными и доступа к ним. Уровень контроля доступа – это уровень, который используется для управления данными и доступа к ним. Логический уровень – это уровень, который используется для управления данными и доступа к ним. Физический уровень – это уровень, который используется для управления данными и доступа к ним.

Внутренняя файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним.

13.4.1 Иерархическая структура каталогов

Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним.

1	...
2	...
3	...
4	...
5	...
6	...
7	...
8	...
9	...
10	...
11	...
12	...
13	...
14	...
15	...
16	...
17	...
18	...
19	...
20	...
21	...
22	...
23	...
24	...
25	...
26	...
27	...
28	...
29	...
30	...
31	...
32	...
33	...
34	...
35	...
36	...
37	...
38	...
39	...
40	...
41	...
42	...
43	...
44	...
45	...
46	...
47	...
48	...
49	...
50	...
51	...
52	...
53	...
54	...
55	...
56	...
57	...
58	...
59	...
60	...
61	...
62	...
63	...
64	...
65	...
66	...
67	...
68	...
69	...
70	...
71	...
72	...
73	...
74	...
75	...
76	...
77	...
78	...
79	...
80	...
81	...
82	...
83	...
84	...
85	...
86	...
87	...
88	...
89	...
90	...
91	...
92	...
93	...
94	...
95	...
96	...
97	...
98	...
99	...
100	...

204 / 287

Осторожно

Вместо переноса на структуры, которые являются не подклассами (показателями), вероятно, имеет место.

— `l_op, l_top` — операции на inode и на файле; `l_sb` — (физический файл доступен только если ФС монтирована).

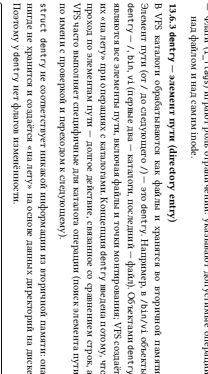


Рис. 43. Структура

– s_blocksize – s_blocksize – размер блока и максимальный размер файла;
– s_magic – магическое число ФС;
– s_type, s_op – тип ФС и функции на суперблоке (super_operations); s_root – точка монтирования (корневой каталог); s_inodes – список всех inode;
– s_mounts – список монтируемых данных ФС (всего их не ФС может быть несколько разных).

209 / 287

13.5. Внутренняя файловая система VFS и Linux

Внутренняя файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним. Файловая система VFS – это набор структур, которые используются для управления данными и доступа к ним.

- 1) Обеспечение доступа к данным;
- 2) Обеспечение доступа к данным;
- 3) Обеспечение доступа к данным;
- 4) Обеспечение доступа к данным;

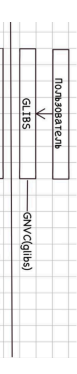


Рис. 44. Внутренняя файловая система VFS

1	...
2	...
3	...
4	...
5	...
6	...
7	...
8	...
9	...
10	...
11	...
12	...
13	...
14	...
15	...
16	...
17	...
18	...
19	...
20	...
21	...
22	...
23	...
24	...
25	...
26	...
27	...
28	...
29	...
30	...
31	...
32	...
33	...
34	...
35	...
36	...
37	...
38	...
39	...
40	...
41	...
42	...
43	...
44	...
45	...
46	...
47	...
48	...
49	...
50	...
51	...
52	...
53	...
54	...
55	...
56	...
57	...
58	...
59	...
60	...
61	...
62	...
63	...
64	...
65	...
66	...
67	...
68	...
69	...
70	...
71	...
72	...
73	...
74	...
75	...
76	...
77	...
78	...
79	...
80	...
81	...
82	...
83	...
84	...
85	...
86	...
87	...
88	...
89	...
90	...
91	...
92	...
93	...
94	...
95	...
96	...
97	...
98	...
99	...
100	...

Рис. 45. Внутренняя файловая система VFS

205 / 287

Осторожно

Вместо переноса на структуры, которые являются не подклассами (показателями), вероятно, имеет место.

категорий и файлы относятся к конкретной скомпьюрированной

13.6.4 file – дескриптор открытого файла

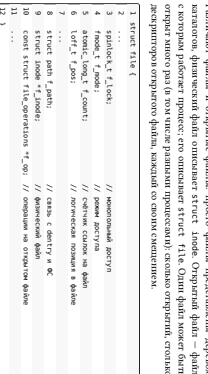


Рис. 46. Внутренняя файловая система VFS

– s_blocksize – s_blocksize – размер блока и максимальный размер файла;
– s_magic – магическое число ФС;
– s_type, s_op – тип ФС и функции на суперблоке (super_operations); s_root – точка монтирования (корневой каталог); s_inodes – список всех inode;
– s_mounts – список монтируемых данных ФС (всего их не ФС может быть несколько разных).

210 / 287

```

1 struct task_struct {
2     #ifdef CONFIG_THREAD_INFO_IN_TASK
3     struct thread_info thread_info;
4     #endif
5     unsigned int    __state;
6     /* saved state for "spinlock sleepers" */
7     unsigned int    saved_state;
8     /*
9     * This begins the randomizable portion of task_struct. Only
10    * scheduling-critical items should be added above here.
11    */
12    randomized_struct_fields_start
13    void*stack;
14    refcount_t       usage;
15    /* Per task flags (PF_*), defined further below: */
16    unsigned int     flags;
17    unsigned int     ptrace;
18    #ifdef CONFIG_MEM_ALLOC_PROFILING
19    struct alloc_tag *alloc_tag;
20    #endif
21    #ifdef CONFIG_SMP
22    int on_cpu;
23    struct __call_single_node    wake_entry;
24    unsigned int     wakee_flips;
25    unsigned long    wakee_flip_decay_ts;
26    struct task_struct *last_wakee;
27    /*
28    * recent_used_cpu is initially set as the last CPU used by a task
29    * that wakes affine another task. Waker/wakee relationships can
30    * push tasks around a CPU where each wakeup moves to the next one.
31    * Tracking a recently used CPU allows a quick search for a recently
32    * used CPU that may be idle.
33    */
34    int recent_used_cpu;
35    int wake_cpu;
36    #endif
37    int on_rq;
38    int prio;
39    int static_prio;
40    int normal_prio;
41    unsigned int     rt_priority;
42    struct sched_entity se;
43    struct sched_rt_entity rt;
44    struct sched_dl_entity dl;
45    struct sched_dl_entity *dl_server;
46    #ifdef CONFIG_SCHED_CLASS_EXT
47    struct sched_ext_entity scx;
48    #endif
49    const struct sched_class    *sched_class;
50    #ifdef CONFIG_SCHED_CORE
51    struct rb_node    core_node;
52    unsigned long     core_cookie;
53    unsigned int      core_occupation;
54    #endif
55    #ifdef CONFIG_CGROUP_SCHED
56    struct task_group *sched_task_group;
57    #endif
58    #ifdef CONFIG_UCLAMP_TASK
59    /*
60    * Clamp values requested for a scheduling entity.
61    * Must be updated with task_rq_lock() held.
62    */
63    struct uclamp_se uclamp_req[UCLAMP_CNT];
64    /*
65    * Effective clamp values used for a scheduling entity.
66    * Must be updated with task_rq_lock() held.
67    */
68    struct uclamp_se uclamp[UCLAMP_CNT];
69    #endif
70    struct sched_statistics    stats;

```



```

71 #ifdef CONFIG_PREEMPT_NOTIFIERS
72 /* List of struct preempt_notifier: */
73 struct hlist_head preempt_notifiers;
74 #endif
75 #ifdef CONFIG_BLK_DEV_IO_TRACE
76 unsigned int    btrace_seq;
77 #endif
78 unsigned int    policy;
79 unsigned long   max_allowed_capacity;
80 int nr_cpus_allowed;
81 const cpumask_t *cpus_ptr;
82 cpumask_t       *user_cpus_ptr;
83 cpumask_t       cpus_mask;
84 void *migration_pending;
85 #ifdef CONFIG_SMP
86 unsigned short  migration_disabled;
87 #endif
88 unsigned short  migration_flags;
89 #ifdef CONFIG_PREEMPT_RCU
90 int rcu_read_lock_nesting;
91 union rcu_special rcu_read_unlock_special;
92 struct list_head rcu_node_entry;
93 struct rcu_node *rcu_blocked_node;
94 #endif /* #ifdef CONFIG_PREEMPT_RCU */
95 #ifdef CONFIG_TASKS_RCU
96 unsigned long   rcu_tasks_nvcsw;
97 u8rcu_tasks_holdout;
98 u8rcu_tasks_idx;
99 int rcu_tasks_idle_cpu;
100 struct list_headrcu_tasks_holdout_list;
101 int rcu_tasks_exit_cpu;
102 struct list_head rcu_tasks_exit_list;
103 #endif /* #ifdef CONFIG_TASKS_RCU */
104 #ifdef CONFIG_TASKS_TRACE_RCU
105 int trc_reader_nesting;
106 int trc_ipi_to_cpu;
107 union rcu_special trc_reader_special;
108 struct list_head trc_holdout_list;
109 struct list_head trc_blkdn_node;
110 int trc_blkdn_cpu;
111 #endif /* #ifdef CONFIG_TASKS_TRACE_RCU */
112 struct sched_info sched_info;
113 struct list_head tasks;
114 #ifdef CONFIG_SMP
115 struct plist_node pushable_tasks;
116 struct rb_node   pushable_dl_tasks;
117 #endif
118 struct mm_struct *mm;
119 struct mm_struct *active_mm;
120 struct address_space *faults_disabled_mapping;
121 int exit_state;
122 int exit_code;
123 int exit_signal;
124 /* The signal sent when the parent dies: */
125 int pdeath_signal;
126 /* JOBCTL_*, siglock protected: */
127 unsigned long   jobctl;
128 /* Used for emulating ABI behavior of previous Linux versions: */
129 unsigned int    personality;
130 /* Scheduler bits, serialized by scheduler locks: */
131 unsigned        sched_reset_on_fork:1;
132 unsigned        sched_contributes_to_load:1;
133 unsigned        sched_migrated:1;
134 unsigned        sched_task_hot:1;
135 /* Force alignment to the next boundary: */
136 unsigned        :0;
137 /* Unserialized, strictly 'current' */
138 /*
139 * This field must not be in the scheduler word above due to wakelist
140 * queueing no longer being serialized by p->on_cpu. However:
141 * p->XXX = X;    ttwu()

```



```

142 * schedule()          if (p->on_rq && ...) // false
143 *   smp_mb__after_spinlock();          if (smp_load_acquire(&p->on_cpu) && //true
144 *   deactivate_task()          ttwu_queue_wakelist())
145 *   p->on_rq = 0;          p->sched_remote_wakeup = Y;
146 * guarantees all stores of 'current' are visible before
147 * ->sched_remote_wakeup gets used, so it can be in this word.
148 */
149 unsigned          sched_remote_wakeup:1;
150 #ifdef CONFIG_RT_MUTEXES
151 unsigned          sched_rt_mutex:1;
152 #endif
153 /* Bit to tell TOMOYO we're in execve(): */
154 unsigned          in_execve:1;
155 unsigned          in_iowait:1;
156 #ifndef TIF_RESTORE_SIGMASK
157 unsigned          restore_sigmask:1;
158 #endif
159 #ifdef CONFIG_MEMCG_V1
160 unsigned          in_user_fault:1;
161 #endif
162 #ifdef CONFIG_LRU_GEN
163 /* whether the LRU algorithm may apply to this access */
164 unsigned          in_lru_fault:1;
165 #endif
166 #ifdef CONFIG_COMPAT_BRK
167 unsigned          brk_randomized:1;
168 #endif
169 #ifdef CONFIG_CGROUPS
170 /* disallow userland-initiated cgroup migration */
171 unsigned          no_cgroup_migration:1;
172 /* task is frozen/stopped (used by the cgroup freezer) */
173 unsigned          frozen:1;
174 #endif
175 #ifdef CONFIG_BLK_CGROUP
176 unsigned          use_memdelay:1;
177 #endif
178 #ifdef CONFIG_PSI
179 /* Stalled due to lack of memory */
180 unsigned          in_memstall:1;
181 #endif
182 #ifdef CONFIG_PAGE_OWNER
183 /* Used by page_owner=on to detect recursion in page tracking. */
184 unsigned          in_page_owner:1;
185 #endif
186 #ifdef CONFIG_EVENTFD
187 /* Recursion prevention for eventfd_signal() */
188 unsigned          in_eventfd:1;
189 #endif
190 #ifdef CONFIG_ARCH_HAS_CPU_PASID
191 unsigned          pasid_activated:1;
192 #endif
193 #ifdef CONFIG_X86_BUS_LOCK_DETECT
194 unsigned          reported_split_lock:1;
195 #endif
196 #ifdef CONFIG_TASK_DELAY_ACCT
197 /* delay due to memory thrashing */
198 unsigned          in_thrashing:1;
199 #endif
200 #ifdef CONFIG_PREEMPT_RT
201 struct netdev_xmitnet_xmit;
202 #endif
203 unsigned long      atomic_flags; /* Flags requiring atomic access. */
204 struct restart_blockrestart_block;
205 pid_t pid;
206 pid_t tgid;
207 #ifdef CONFIG_STACKPROTECTOR
208 /* Canary value for the -fstack-protector GCC feature: */
209 unsigned long      stack_canary;
210 #endif
211 /*
212 * Point ers to the (original) parent process, youngest child, younger sibling,

```



```

213 * older sibling, respectively. (p->father can be replaced with
214 * p->real_parent->pid)
215 */
216 /* Real parent process: */
217 struct task_struct __rcu      *real_parent;
218 /* Recipient of SIGCHLD, wait4() reports: */
219 struct task_struct __rcu      *parent;
220 /*
221 * Children/sibling form the list of natural children:
222 */
223 struct list_head children;
224 struct list_head sibling;
225 struct task_struct *group_leader;
226 /*
227 * 'ptraced' is the list of tasks this task is using ptrace() on.
228 * This includes both natural children and PTRACE_ATTACH targets.
229 * 'ptrace_entry' is this task's link on the p->parent->ptraced list.
230 */
231 struct list_head ptraced;
232 struct list_head ptrace_entry;
233 /* PID/PID hash table linkage. */
234 struct pid      *thread_pid;
235 struct hlist_node pid_links[PIDTYPE_MAX];
236 struct list_head thread_node;
237 struct completion *vfork_done;
238 /* CLONE_CHILD_SETTID: */
239 int __user      *set_child_tid;
240 /* CLONE_CHILD_CLEARTID: */
241 int __user      *clear_child_tid;
242 /* PF_KTHREAD | PF_IO_WORKER */
243 void * worker_private;
244 u64 utime;
245 u64 stime;
246 #ifdef CONFIG_ARCH_HAS_SCALED_CPUTIME
247 u64 utimescaled;
248 u64 stimescaled;
249 #endif
250 u64 gtime;
251 struct prev_cpu timeprev_cputime;
252 #ifdef CONFIG_VIRT_CPU_ACCOUNTING_GEN
253 struct vtime      vtime;
254 #endif
255 #ifdef CONFIG_NO_HZ_FULL
256 atomic_t          tick_dep_mask;
257 #endif
258 /* Context switch counts: */
259 unsigned long     nvcs;
260 unsigned long     nivcs;
261 /* Monotonic time in nsecs: */
262 u64 start_time;
263 /* Boot based time in nsecs: */
264 u64 start_boottime;
265 /* MM fault and swap info: this can arguably be seen as either mm-specific or
thread-specific: */
266 unsigned long     min_flt;
267 unsigned long     maj_flt;
268 /* Empty if CONFIG_POSIX_CPUTIMERS=n */
269 struct posix_cputimers posix_cputimers;
270 #ifdef CONFIG_POSIX_CPU_TIMERS_TASK_WORK
271 struct posix_cputimers_work posix_cputimers_work;
272 #endif
273 /* Process credentials: */
274 /* Tracer's credentials at attach: */
275 const struct cred __rcu *ptracer_cred;
276 /* Objective and real subjective task credentials (COW): */
277 const struct cred __rcu *real_cred;
278 /* Effective (overridable) subjective task credentials (COW): */
279 const struct cred __rcu *cred;
280 #ifdef CONFIG_KEYS
281 /* Cached requested key. */
282 struct key          *cached_requested_key;

```



```

283 #endif
284 /*
285  * executable name, excluding path.
286  * - normally initialized begin_new_exec()
287  * - set it with set_task_comm()
288  * - strncpy_pad() to ensure it is always NUL-terminated and zero-padded
289  * - task_lock() to ensure the operation is atomic and the name is fully updated.
290  */
291 char comm[TASK_COMM_LEN];
292 struct nameidata*nameidata;
293 #ifdef CONFIG_SYSVIPC
294 struct sysv_sem sysvsem;
295 struct sysv_shm sysvshm;
296 #endif
297 #ifdef CONFIG_DETECT_HUNG_TASK
298 unsigned long last_switch_count;
299 unsigned long last_switch_time;
300 #endif
301 /* Filesystem information: */
302 struct fs_struct *fs;
303 /* Open file information: */
304 struct files_struct *files;
305 #ifdef CONFIG_IO_URING
306 struct io_uring_task*io_uring;
307 #endif
308 /* Namespaces: */
309 struct nsproxy *nsproxy;
310 /* Signal handlers: */
311 struct signal_struct *signal;
312 struct sighand_struct __rcu*sighand;
313 sigset_t blocked;
314 sigset_t real_blocked;
315 /* Restored if set_restore_sigmask() was used: */
316 sigset_t saved_sigmask;
317 struct sigpendingpending;
318 unsigned long sas_ss_sp;
319 size_t sas_ss_size;
320 unsigned int sas_ss_flags;
321 struct callback_head *task_works;
322 #ifdef CONFIG_AUDIT
323 #ifdef CONFIG_AUDITSYSCALL
324 struct audit_context *audit_context;
325 #endif
326 kuid_t loginuid;
327 unsigned int sessionid;
328 #endif
329 struct seccomp seccomp;
330 struct syscall_user_dispatch syscall_dispatch;
331 /* Thread group tracking: */
332 u64 parent_exec_id;
333 u64 self_exec_id;
334 /* Protection against (de-)allocation: mm, files, fs, tty, keyrings, mems_allowed,
335 mempolicy: */
336 spinlock_t alloc_lock;
337 /* Protection of the PI data structures: */
338 raw_spinlock_t pi_lock;
339 struct wake_q_nodewake_q;
340 #ifdef CONFIG_RT_MUTEXES
341 /* PI waiters blocked on a rt_mutex held by this task: */
342 struct rb_root_cached pi_waiters;
343 /* Updated under owner's pi_lock and rq lock */
344 struct task_struct *pi_top_task;
345 /* Deadlock detection and priority inheritance handling: */
346 struct rt_mutex_waiter *pi_blocked_on;
347 #endif
348 #ifdef CONFIG_DEBUG_MUTEXES
349 /* Mutex deadlock detection: */
350 struct mutex_waiter *blocked_on;
351 #endif
352 #ifdef CONFIG_DETECT_HUNG_TASK_BLOCKER
353 struct mutex *blocker_mutex;

```

```

176         return b;
177     }
178
179     static const proc_desc_t proc_desc_0 = {
180         .proc_name = "fortran",
181         .proc_desc = "fortran",
182         .proc_arch = "fortran",
183         .proc_vendor = "fortran",
184         .proc_release = "fortran.release",
185     };
186
187     static const proc_desc_t proc_desc_1 = {
188         .proc_name = "coq",
189         .proc_desc = "coq",
190         .proc_arch = "coq",
191         .proc_vendor = "coq",
192         .proc_release = "coq.release",
193     };
194
195     static const proc_desc_t proc_desc_2 = {
196         .proc_name = "coq",
197         .proc_desc = "coq",
198         .proc_arch = "coq",
199         .proc_vendor = "coq",
200         .proc_release = "coq.release",
201     };
202
203     static const proc_desc_t proc_desc_3 = {
204         .proc_name = "coq",
205         .proc_desc = "coq",
206         .proc_arch = "coq",
207         .proc_vendor = "coq",
208         .proc_release = "coq.release",
209     };
210
211     static const proc_desc_t proc_desc_4 = {
212         .proc_name = "coq",
213         .proc_desc = "coq",
214         .proc_arch = "coq",
215         .proc_vendor = "coq",
216         .proc_release = "coq.release",
217     };
218
219     static const proc_desc_t proc_desc_5 = {
220         .proc_name = "coq",
221         .proc_desc = "coq",
222         .proc_arch = "coq",
223         .proc_vendor = "coq",
224         .proc_release = "coq.release",
225     };
226
227     static const proc_desc_t proc_desc_6 = {
228         .proc_name = "coq",
229         .proc_desc = "coq",
230         .proc_arch = "coq",
231         .proc_vendor = "coq",
232         .proc_release = "coq.release",
233     };
234
235     static const proc_desc_t proc_desc_7 = {
236         .proc_name = "coq",
237         .proc_desc = "coq",
238         .proc_arch = "coq",
239         .proc_vendor = "coq",
240         .proc_release = "coq.release",
241     };
242
243     static const proc_desc_t proc_desc_8 = {
244         .proc_name = "coq",
245         .proc_desc = "coq",
246         .proc_arch = "coq",
247         .proc_vendor = "coq",
248         .proc_release = "coq.release",
249     };
250
251     static const proc_desc_t proc_desc_9 = {
252         .proc_name = "coq",
253         .proc_desc = "coq",
254         .proc_arch = "coq",
255         .proc_vendor = "coq",
256         .proc_release = "coq.release",
257     };
258
259     static const proc_desc_t proc_desc_10 = {
260         .proc_name = "coq",
261         .proc_desc = "coq",
262         .proc_arch = "coq",
263         .proc_vendor = "coq",
264         .proc_release = "coq.release",
265     };
266
267     static const proc_desc_t proc_desc_11 = {
268         .proc_name = "coq",
269         .proc_desc = "coq",
270         .proc_arch = "coq",
271         .proc_vendor = "coq",
272         .proc_release = "coq.release",
273     };
274
275     static const proc_desc_t proc_desc_12 = {
276         .proc_name = "coq",
277         .proc_desc = "coq",
278         .proc_arch = "coq",
279         .proc_vendor = "coq",
280         .proc_release = "coq.release",
281     };
282
283     static const proc_desc_t proc_desc_13 = {
284         .proc_name = "coq",
285         .proc_desc = "coq",
286         .proc_arch = "coq",
287         .proc_vendor = "coq",
288         .proc_release = "coq.release",
289     };
290
291     static const proc_desc_t proc_desc_14 = {
292         .proc_name = "coq",
293         .proc_desc = "coq",
294         .proc_arch = "coq",
295         .proc_vendor = "coq",
296         .proc_release = "coq.release",
297     };
298
299     static const proc_desc_t proc_desc_15 = {
300         .proc_name = "coq",
301         .proc_desc = "coq",
302         .proc_arch = "coq",
303         .proc_vendor = "coq",
304         .proc_release = "coq.release",
305     };
306
307     static const proc_desc_t proc_desc_16 = {
308         .proc_name = "coq",
309         .proc_desc = "coq",
310         .proc_arch = "coq",
311         .proc_vendor = "coq",
312         .proc_release = "coq.release",
313     };
314
315     static const proc_desc_t proc_desc_17 = {
316         .proc_name = "coq",
317         .proc_desc = "coq",
318         .proc_arch = "coq",
319         .proc_vendor = "coq",
320         .proc_release = "coq.release",
321     };
322
323     static const proc_desc_t proc_desc_18 = {
324         .proc_name = "coq",
325         .proc_desc = "coq",
326         .proc_arch = "coq",
327         .proc_vendor = "coq",
328         .proc_release = "coq.release",
329     };
330
331     static const proc_desc_t proc_desc_19 = {
332         .proc_name = "coq",
333         .proc_desc = "coq",
334         .proc_arch = "coq",
335         .proc_vendor = "coq",
336         .proc_release = "coq.release",
337     };
338
339     static const proc_desc_t proc_desc_20 = {
340         .proc_name = "coq",
341         .proc_desc = "coq",
342         .proc_arch = "coq",
343         .proc_vendor = "coq",
344         .proc_release = "coq.release",
345     };
346
347     static const proc_desc_t proc_desc_21 = {
348         .proc_name = "coq",
349         .proc_desc = "coq",
350         .proc_arch = "coq",
351         .proc_vendor = "coq",
352         .proc_release = "coq.release",
353     };
354
355     static const proc_desc_t proc_desc_22 = {
356         .proc_name = "coq",
357         .proc_desc = "coq",
358         .proc_arch = "coq",
359         .proc_vendor = "coq",
360         .proc_release = "coq.release",
361     };
362
363     static const proc_desc_t proc_desc_23 = {
364         .proc_name = "coq",
365         .proc_desc = "coq",
366         .proc_arch = "coq",
367         .proc_vendor = "coq",
368         .proc_release = "coq.release",
369     };
370
371     static const proc_desc_t proc_desc_24 = {
372         .proc_name = "coq",
373         .proc_desc = "coq",
374         .proc_arch = "coq",
375         .proc_vendor = "coq",
376         .proc_release = "coq.release",
377     };
378
379     static const proc_desc_t proc_desc_25 = {
380         .proc_name = "coq",
381         .proc_desc = "coq",
382         .proc_arch = "coq",
383         .proc_vendor = "coq",
384         .proc_release = "coq.release",
385     };
386
387     static const proc_desc_t proc_desc_26 = {
388         .proc_name = "coq",
389         .proc_desc = "coq",
390         .proc_arch = "coq",
391         .proc_vendor = "coq",
392         .proc_release = "coq.release",
393     };
394
395     static const proc_desc_t proc_desc_27 = {
396         .proc_name = "coq",
397         .proc_desc = "coq",
398         .proc_arch = "coq",
399         .proc_vendor = "coq",
400         .proc_release = "coq.release",
401     };
402
403     static const proc_desc_t proc_desc_28 = {
404         .proc_name = "coq",
405         .proc_desc = "coq",
406         .proc_arch = "coq",
407         .proc_vendor = "coq",
408         .proc_release = "coq.release",
409     };
410
411     static const proc_desc_t proc_desc_29 = {
412         .proc_name = "coq",
413         .proc_desc = "coq",
414         .proc_arch = "coq",
415         .proc_vendor = "coq",
416         .proc_release = "coq.release",
417     };
418
419     static const proc_desc_t proc_desc_30 = {
420         .proc_name = "coq",
421         .proc_desc = "coq",
422         .proc_arch = "coq",
423         .proc_vendor = "coq",
424         .proc_release = "coq.release",
425     };
426
427     static const proc_desc_t proc_desc_31 = {
428         .proc_name = "coq",
429         .proc_desc = "coq",
430         .proc_arch = "coq",
431         .proc_vendor = "coq",
432         .proc_release = "coq.release",
433     };
434
435     static const proc_desc_t proc_desc_32 = {
436         .proc_name = "coq",
437         .proc_desc = "coq",
438         .proc_arch = "coq",
439         .proc_vendor = "coq",
440         .proc_release = "coq.release",
441     };
442
443     static const proc_desc_t proc_desc_33 = {
444         .proc_name = "coq",
445         .proc_desc = "coq",
446         .proc_arch = "coq",
447         .proc_vendor = "coq",
448         .proc_release = "coq.release",
449     };
450
451     static const proc_desc_t proc_desc_34 = {
452         .proc_name = "coq",
453         .proc_desc = "coq",
454         .proc_arch = "coq",
455         .proc_vendor = "coq",
456         .proc_release = "coq.release",
457     };
458
459     static const proc_desc_t proc_desc_35 = {
460         .proc_name = "coq",
461         .proc_desc = "coq",
462         .proc_arch = "coq",
463         .proc_vendor = "coq",
464         .proc_release = "coq.release",
465     };
466
467     static const proc_desc_t proc_desc_36 = {
468         .proc_name = "coq",
469         .proc_desc = "coq",
470         .proc_arch = "coq",
471         .proc_vendor = "coq",
472         .proc_release = "coq.release",
473     };
474
475     static const proc_desc_t proc_desc_37 = {
476         .proc_name = "coq",
477         .proc_desc = "coq",
478         .proc_arch = "coq",
479         .proc_vendor = "coq",
480         .proc_release = "coq.release",
481     };
482
483     static const proc_desc_t proc_desc_38 = {
484         .proc_name = "coq",
485         .proc_desc = "coq",
486         .proc_arch = "coq",
487         .proc_vendor = "coq",
488         .proc_release = "coq.release",
489     };
490
491     static const proc_desc_t proc_desc_39 = {
492         .proc_name = "coq",
493         .proc_desc = "coq",
494         .proc_arch = "coq",
495         .proc_vendor = "coq",
496         .proc_release = "coq.release",
497     };
498
499     static const proc_desc_t proc_desc_40 = {
500         .proc_name = "coq",
501         .proc_desc = "coq",
502         .proc_arch = "coq",
503         .proc_vendor = "coq",
504         .proc_release = "coq.release",
505     };
506
507     static const proc_desc_t proc_desc_41 = {
508         .proc_name = "coq",
509         .proc_desc = "coq",
510         .proc_arch = "coq",
511         .proc_vendor = "coq",
512         .proc_release = "coq.release",
513     };
514
515     static const proc_desc_t proc_desc_42 = {
516         .proc_name = "coq",
517         .proc_desc = "coq",
518         .proc_arch = "coq",
519         .proc_vendor = "coq",
520         .proc_release = "coq.release",
521     };
522
523     static const proc_desc_t proc_desc_43 = {
524         .proc_name = "coq",
525         .proc_desc = "coq",
526         .proc_arch = "coq",
527         .proc_vendor = "coq",
528         .proc_release = "coq.release",
529
```

251 / 287

```

120 static void _exit(fortune_exit_t) {
121     proc_remove(proc_splink.entry);
122     proc_remove(proc_fila);
123     proc_remove(proc_dirt);
124     vfree(cookie_buf);
125     printf(KERN_INFO "fortune module unloaded!\n");
126 }
127
128 module_init(fortune_init);
129 module_exit(fortune_exit);

```

252 / 287

1. O_2 (aq) + H^+ (aq) $\rightleftharpoons$ H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 2. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 3. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 4. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 5. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 6. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 7. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 8. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 9. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)
 10. O_2 (aq) + H_2O_2 (aq) $\rightleftharpoons$ H_2O (l) + O (g)

Вместе с `0_CREAT` часто используется флаг `0_EXCL`: он заставляет систему проверить, существует ли уже создаваемый файл, чтобы не потерять ранее записанную в него информацию. Если файла с таким именем уже есть, вызов `open` с `0_CREAT | 0_EXCL` завершается ошибкой.

Ипрмсп

Вопросы ЖП

[illegible]

СОДЕРЖАНИЕ

2 / 30

Као државно

[illegible]

Показатели эффективности деятельности предприятия (табл. 1) (показатели эффективности деятельности предприятия) – это показатели, характеризующие деятельность предприятия с точки зрения его финансового, производственного, социального, экологического и иного благополучия. Показатели эффективности деятельности предприятия являются основой для разработки стратегии предприятия, а также для оценки его деятельности. Показатели эффективности деятельности предприятия являются основой для разработки стратегии предприятия, а также для оценки его деятельности.

16.2 Системный вызов open(): смысл параметров

филь, что нужно открыть системным вызовом open(). В простейшем случае пользователи могут использовать следующие формулы вызова:

- pathname – имя отработанного файла;
- flags – битовый код определяющий режим доступа и поведение при открытии;
- mode – права доступа создаваемого файла.

файла, тогда нужно указать права доступа для трёх групп пользователей. Если же опере выданы для открытия существующего файла, mode не нужен. При создании нового файла создаётся новый inode (инициализируется структура inode).

[illegible]

```

353 #endif
354 #ifdef CONFIG_DEBUG_ATOMIC_SLEEP
355 int non_block_count;
356 #endif
357 #ifdef CONFIG_TRACE_IRQFLAGS
358 struct irqtrace_eventsirqtrace;
359 unsigned int    hardirq_threaded;
360 u64 hardirq_chain_key;
361 int softirqs_enabled;
362 int softirq_context;
363 int irq_config;
364 #endif
365 #ifdef CONFIG_PREEMPT_RT
366 int softirq_disable_cnt;
367 #endif
368 #ifdef CONFIG_LOCKDEP
369 # define MAX_LOCK_DEPTH 48UL
370 u64 curr_chain_key;
371 int lockdep_depth;
372 unsigned int    lockdep_recursion;
373 struct held_lockheld_locks[MAX_LOCK_DEPTH];
374 #endif
375 #if defined(CONFIG_UBSAN) && !defined(CONFIG_UBSAN_TRAP)
376 unsigned int    in_ubsan;
377 #endif
378 /* Journalling filesystem info: */
379 void * journal_info;
380 /* Stacked block device info: */
381 struct bio_list *bio_list;
382 /* Stack plugging: */
383 struct blk_plug *plug;
384 /* VM state: */
385 struct reclaim_state*reclaim_state;
386 struct io_context*io_context;
387 #ifdef CONFIG_COMPACTION
388 struct capture_control*capture_control;
389 #endif
390 /* Ptrace state: */
391 unsigned long    ptrace_message;
392 kernel_siginfo_t*last_siginfo;
393 struct task_io_accounting    ioac;
394 #ifdef CONFIG_PSI
395 /* Pressure stall state */
396 unsigned int    psi_flags;
397 #endif
398 #ifdef CONFIG_TASK_XACCT
399 /* Accumulated RSS usage: */
400 u64 acct_rss_mem1;
401 /* Accumulated virtual memory usage: */
402 u64 acct_vm_mem1;
403 /* stime + utime since last update: */
404 u64 acct_timexpd;
405 #endif
406 #ifdef CONFIG_CPUSETS
407 /* Protected by ->alloc_lock: */
408 nodemask_t    mems_allowed;
409 /* Sequence number to catch updates: */
410 seqcount_spinlock_tmems_allowed_seq;
411 int cpuset_mem_spread_rotor;
412 #endif
413 #ifdef CONFIG_CGROUPS
414 /* Control Group info protected by css_set_lock: */
415 struct css_set __rcu *cgroups;
416 /* cg_list protected by css_set_lock and tsk->alloc_lock: */
417 struct list_head cg_list;
418 #endif
419 #ifdef CONFIG_X86_CPU_RESCTRL
420 u32 closid;
421 u32 rmid;
422 #endif
423 #ifdef CONFIG_FUTEX

```



```

424 struct robust_list_head __user *robust_list;
425 #ifdef CONFIG_COMPAT
426 struct compat_robust_list_head __user *compat_robust_list;
427 #endif
428 struct list_head pi_state_list;
429 struct futex_pi_state *pi_state_cache;
430 struct mutex futex_exit_mutex;
431 unsigned int futex_state;
432 #endif
433 #ifdef CONFIG_PERF_EVENTS
434 u8perf_recursion[PERF_NR_CONTEXTS];
435 struct perf_event_context *perf_event_ctxp;
436 struct mutex perf_event_mutex;
437 struct list_headperf_event_list;
438 struct perf_ctx_data __rcu *perf_ctx_data;
439 #endif
440 #ifdef CONFIG_DEBUG_PREEMPT
441 unsigned long preempt_disable_ip;
442 #endif
443 #ifdef CONFIG_NUMA
444 /* Protected by alloc_lock: */
445 struct mempolicy*mempolicy;
446 short il_prev;
447 u8 il_weight;
448 short pref_node_fork;
449 #endif
450 #ifdef CONFIG_NUMA_BALANCING
451 int numa_scan_seq;
452 unsigned int numa_scan_period;
453 unsigned int numa_scan_period_max;
454 int numa_preferred_nid;
455 unsigned long numa_migrate_retry;
456 /* Migration stamp: */
457 u64 node_stamp;
458 u64 last_task_numa_placement;
459 u64 last_sum_exec_runtime;
460 struct callback_headnuma_work;
461 /*
462  * This point er is only modified for current in syscall and
463  * pagefault context (and for tasks being destroyed), so it can be read
464  * from any of the following contexts:
465  * - RCU read-side critical section
466  * - current->numa_group from everywhere
467  * - task's runqueue locked, task not running
468  */
469 struct numa_group __rcu* numa_group;
470 /*
471  * numa_faults is an array split into four regions:
472  * faults_memory, faults_cpu, faults_memory_buffer, faults_cpu_buffer
473  * in this precise order.
474  * faults_memory: Exponential decaying average of faults on a per-node
475  * basis. Scheduling placement decisions are made based on these
476  * counts. The values remain static for the duration of a PTE scan.
477  * faults_cpu: Track the nodes the process was running on when a NUMA
478  * hinting fault was incurred.
479  * faults_memory_buffer and faults_cpu_buffer: Record faults per node
480  * during the current scan window. When the scan completes, the counts
481  * in faults_memory and faults_cpu decay and these values are copied.
482  */
483 unsigned long *numa_faults;
484 unsigned long total_numa_faults;
485 /*
486  * numa_faults_locality tracks if faults recorded during the last
487  * scan window were remote/local or failed to migrate. The task scan
488  * period is adapted based on the locality of the faults with different
489  * weights depending on whether they were shared or private faults
490  */
491 unsigned long numa_faults_locality[3];
492 unsigned long numa_pages_migrated;
493 #endif /* CONFIG_NUMA_BALANCING */
494 #ifdef CONFIG_RSEQ

```



```

495 struct rseq __user *rseq;
496 u32 rseq_len;
497 u32 rseq_sig;
498 /*
499  * RmW on rseq_event_mask must be performed atomically
500  * with respect to preemption.
501  */
502 unsigned long rseq_event_mask;
503 #ifdef CONFIG_DEBUG_RSEQ
504 /*
505  * This is a place holder to save a copy of the rseq fields for
506  * validation of read-only fields. The struct rseq has a
507  * variable-length array at the end, so it cannot be used
508  * directly. Reserve a size large enough for the known fields.
509  */
510 char rseq_fields[sizeof(struct rseq)];
511 #endif
512 #endif
513 #ifdef CONFIG_SCHED_MM_CID
514 int mm_cid; /* Current cid in mm */
515 int last_mm_cid; /* Most recent cid in mm */
516 int migrate_from_cpu;
517 int mm_cid_active; /* Whether cid bitmap is active */
518 struct callback_headcid_work;
519 #endif
520 struct tlbflush_unmap_batch tlb_ubic;
521 /* Cache last used pipe for splice(): */
522 struct pipe_inode_info*splice_pipe;
523 struct page_fragtask_frag;
524 #ifdef CONFIG_TASK_DELAY_ACCT
525 struct task_delay_info*delays;
526 #endif
527 #ifdef CONFIG_FAULT_INJECTION
528 int make_it_fail;
529 unsigned int fail_nth;
530 #endif
531 /*
532  * When (nr_dirtied >= nr_dirtied_pause), it's time to call
533  * balance_dirty_pages() for a dirty throttling pause:
534  */
535 int nr_dirtied;
536 int nr_dirtied_pause;
537 /* Start of a write-and-pause period: */
538 unsigned long dirty_paused_when;
539 #ifdef CONFIG_LATENCYTOP
540 int latency_record_count;
541 struct latency_recordlatency_record[LT_SAVECOUNT];
542 #endif
543 /*
544  * Time slack values; these are used to round up poll() and
545  * select() etc timeout values. These are in nanoseconds.
546  */
547 u64 timer_slack_ns;
548 u64 default_timer_slack_ns;
549 #if defined(CONFIG_KASAN_GENERIC) || defined(CONFIG_KASAN_SW_TAGS)
550 unsigned int kasan_depth;
551 #endif
552 #ifdef CONFIG_KCSAN
553 struct kcsan_ctxkcsan_ctx;
554 #ifdef CONFIG_TRACE_IRQFLAGS
555 struct irqtrace_eventskcsan_save_irqtrace;
556 #endif
557 #ifdef CONFIG_KCSAN_WEAK_MEMORY
558 int kcsan_stack_depth;
559 #endif
560 #endif
561 #ifdef CONFIG_KMSAN
562 struct kmsan_ctxkmsan_ctx;
563 #endif
564 #if IS_ENABLED(CONFIG_KUNIT)
565 struct kunit *kunit_test;

```



```

566 #endif
567 #ifndef CONFIG_FUNCTION_GRAPH_TRACER
568 /* Index of current stored address in ret_stack: */
569 int curr_ret_stack;
570 int curr_ret_depth;
571 /* Stack of return addresses for return function tracing: */
572 unsigned long *ret_stack;
573 /* Timestamp for last schedule: */
574 unsigned long longftrace_timestamp;
575 unsigned long longftrace_sleeptime;
576 /*
577  * Number of functions that haven't been traced
578  * because of depth overrun:
579  */
580 atomic_t trace_overrun;
581 /* Pause tracing: */
582 atomic_t tracing_graph_pause;
583 #endif
584 #ifdef CONFIG_TRACING
585 /* Bitmask and counter of trace recursion: */
586 unsigned long trace_recursion;
587 #endif /* CONFIG_TRACING */
588 #ifdef CONFIG_KCOV
589 /* See kernel/kcov.c for more details. */
590 /* Coverage collection mode enabled for this task (0 if disabled): */
591 unsigned int kcov_mode;
592 /* Size of the kcov_area: */
593 unsigned int kcov_size;
594 /* Buffer for coverage collection: */
595 void *kcov_area;
596 /* KCOV descriptor wired with this task or NULL: */
597 struct kcov *kcov;
598 /* KCOV common handle for remote coverage collection: */
599 u64 kcov_handle;
600 /* KCOV sequence number: */
601 int kcov_sequence;
602 /* Collect coverage from softirq context: */
603 unsigned int kcov_softirq;
604 #endif
605 #ifdef CONFIG_MEMCG_V1
606 struct mem_cgroup *memcg_in_oom;
607 #endif
608 #ifdef CONFIG_MEMCG
609 /* Number of pages to reclaim on returning to userland: */
610 unsigned int memcg_nr_pages_over_high;
611 /* Used by memcontrol for targeted memcg charge: */
612 struct mem_cgroup*active_memcg;
613 /* Cache for current->cgroups->memcg->objcg lookups: */
614 struct obj_cgroup *objcg;
615 #endif
616 #ifdef CONFIG_BLK_CGROUP
617 struct gendisk *throttle_disk;
618 #endif
619 #ifdef CONFIG_UPROBES
620 struct uprobe_task *utask;
621 #endif
622 #if defined(CONFIG_BCACHE) || defined(CONFIG_BCACHE_MODULE)
623 unsigned int sequential_io;
624 unsigned int sequential_io_avg;
625 #endif
626 struct kmap_ctrlkmap_ctrl;
627 #ifdef CONFIG_DEBUG_ATOMIC_SLEEP
628 unsigned long task_state_change;
629 # ifdef CONFIG_PREEMPT_RT
630 unsigned long saved_state_change;
631 # endif
632 #endif
633 struct rcu_head rcu;
634 refcount_t rcu_users;
635 int pagefault_disabled;
636 #ifdef CONFIG_MMU

```

```

637 struct task_struct *oom_reaper_list;
638 struct timer_list oom_reaper_timer;
639 #endif
640 #ifdef CONFIG_VMAP_STACK
641 struct vm_struct *stack_vm_area;
642 #endif
643 #ifdef CONFIG_THREAD_INFO_IN_TASK
644 /* A live task holds one reference: */
645 refcount_t      stack_refcount;
646 #endif
647 #ifdef CONFIG_LIVEPATCH
648 int patch_state;
649 #endif
650 #ifdef CONFIG_SECURITY
651 /* Used by LSM modules for access restriction: */
652 void * security;
653 #endif
654 #ifdef CONFIG_BPF_SYSCALL
655 /* Used by BPF task local storage */
656 struct bpf_local_storage __rcu *bpf_storage;
657 /* Used for BPF run context */
658 struct bpf_run_ctx*bpf_ctx;
659 #endif
660 /* Used by BPF for per-TASK xdp storage */
661 struct bpf_net_context*bpf_net_context;
662 #ifdef CONFIG_GCC_PLUGIN_STACKLEAK
663 unsigned long lowest_stack;
664 unsigned long prev_lowest_stack;
665 #endif
666 #ifdef CONFIG_X86_MCE
667 void __user *mce_vaddr;
668 __u64 mce_kflags;
669 u64 mce_addr;
670 __u64 mce_ripv : 1,
671 mce_whole_page : 1,
672 __mce_reserved : 62;
673 struct callback_headmce_kill_me;
674 int mce_count;
675 #endif
676 #ifdef CONFIG_KRETPROBES
677 struct llist_head kretprobe_instances;
678 #endif
679 #ifdef CONFIG_RETHOOK
680 struct llist_head rethooks;
681 #endif
682 #ifdef CONFIG_ARCH_HAS_PARANOID_L1D_FLUSH
683 /*
684  * If L1D flush is supported on mm context switch
685  * then we use this callback head to queue kill work
686  * to kill tasks that are not running on SMT disabled
687  * cores
688  */
689 struct callback_headl1d_flush_kill;
690 #endif
691 #ifdef CONFIG_RV
692 /*
693  * Per-task RV monitor. Nowadays fixed in RV_PER_TASK_MONITORS.
694  * If we find justification for more monitors, we can think
695  * about adding more or developing a dynamic method. So far,
696  * none of these are justified.
697  */
698 union rv_task_monitorrv[RV_PER_TASK_MONITORS];
699 #endif
700 #ifdef CONFIG_USER_EVENTS
701 struct user_event_mm *user_event_mm;
702 #endif
703 randomized_struct_fields_end
704 /* CPU-specific state of this task: */
705 struct thread_struct thread;
706 };

```